Orchestration et Kubernetes

Maxime El Haddari

October 28, 2026

1. Introduction
2. Pourquoi Kubernetes ?
3. Kubernetes: les concepts
4. Les ressources Kubernetes
5. Best Practices
6. Kubernetes avancé
7. Helm
8. GitOps

Quiz:

- Votre application fonctionne très bien sur votre machine. Et ensuite ?

Quiz:

- Votre application fonctionne très bien sur votre machine. Et ensuite ?
- Que se passe-t-il quand 10, 1 000 puis 100 000 utilisateurs arrivent en même temps ?

Quiz:

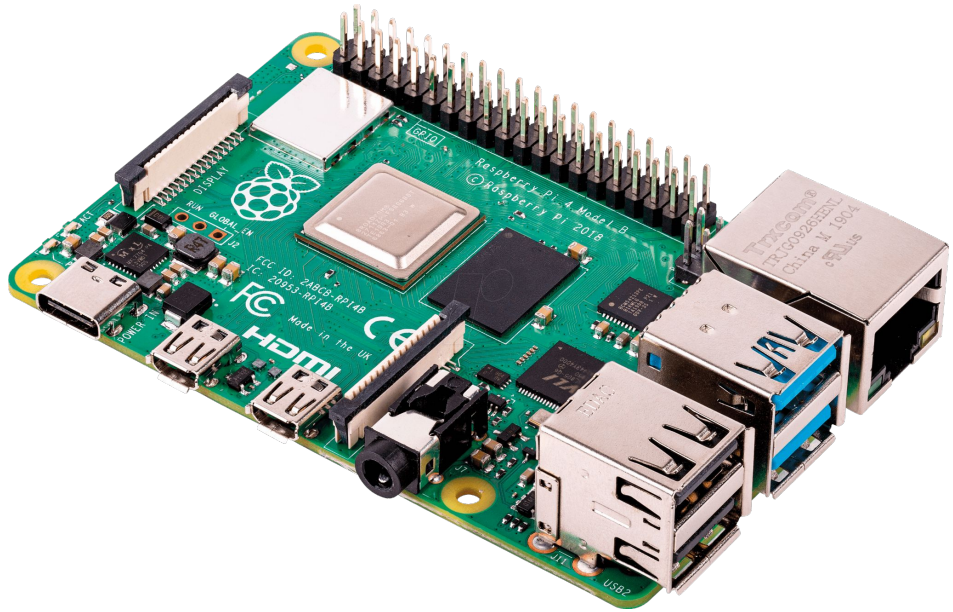
- Votre application fonctionne très bien sur votre machine. Et ensuite ?
- Que se passe-t-il quand 10, 1 000 puis 100 000 utilisateurs arrivent en même temps ?
- Qu'est-ce qui casse en premier ?

Quiz:

- Votre application fonctionne très bien sur votre machine. Et ensuite ?
- Que se passe-t-il quand 10, 1 000 puis 100 000 utilisateurs arrivent en même temps ?
- Qu'est-ce qui casse en premier ?
- Comment on répare ? Et qui se lève la nuit pour le faire ?

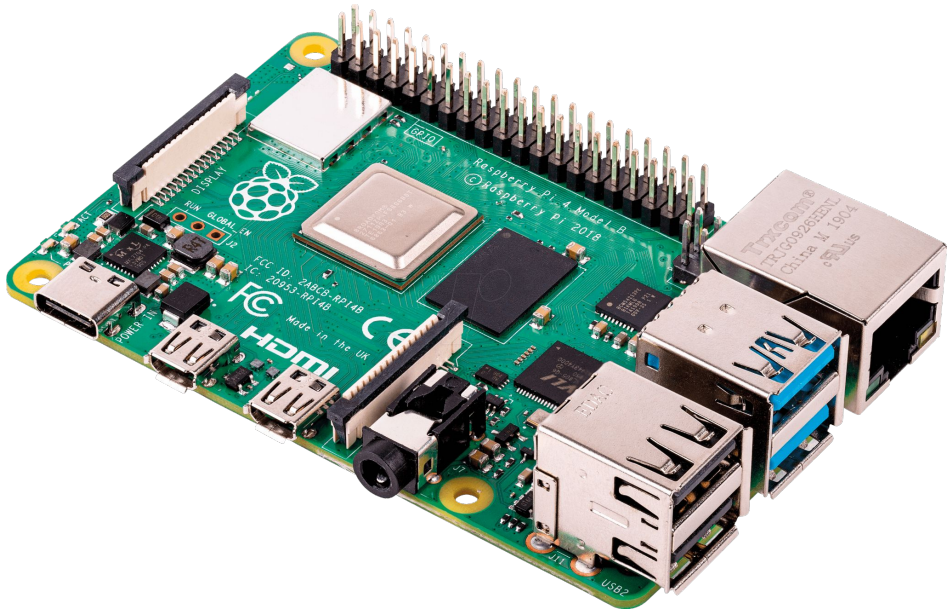
Mon service tourne sur un Raspberry Pi 5 :

- CPU: 2,4 GHz quad-core 64-bit Arm
- RAM: 16 Go
- Stockage: carte SD de 256 Go
- Bande passante: 1 Gb/s



Mon service tourne sur un Raspberry Pi 5 :

- CPU: 2,4 GHz quad-core 64-bit Arm
- RAM: 16 Go
- Stockage: carte SD de 256 Go
- Bande passante: 1 Gb/s



Puis mon service devient populaire...

`too many connections`

Pas de souci, on prend une plus grosse machine (Scale-a2) :

- CPU: 24 cores / 48 threads - 2,9 GHz / 3,9 GHz
- RAM: 512 Go
- Stockage: 2 x 1,92 To



Pas de souci, on prend une plus grosse machine (Scale-a2) :

- CPU: 24 cores / 48 threads - 2,9 GHz / 3,9 GHz
- RAM: 512 Go
- Stockage: 2 x 1,92 To

Et pourtant...

```
FATAL: remaining connection slots are reserved for non-replication superuser connections
```



Scale vertical vs scale horizontal :

Vertical (*"scale up"*) :

- On grossit la machine
- Simple, rien à changer dans le code
- Mais il y a un plafond (et un prix)
- Toujours un *"Single Point Of Failure"*
- Souvent un redémarrage pour changer de taille

Horizontal (*"scale out"*) :

Scale vertical vs scale horizontal :

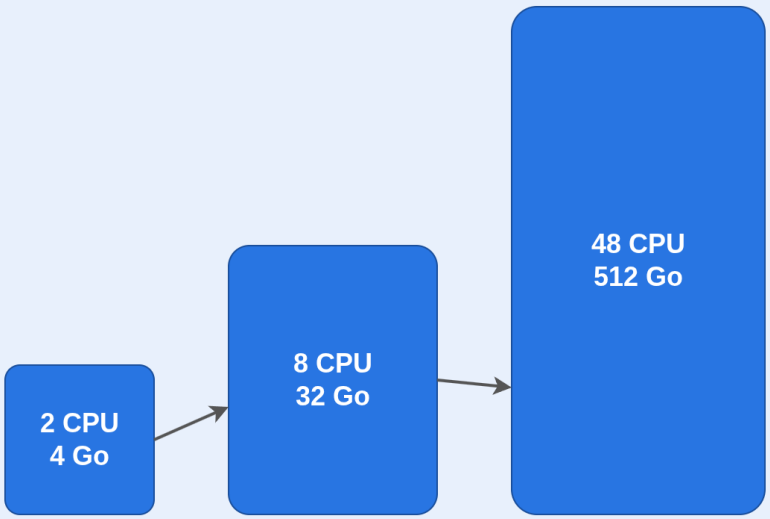
Vertical (*"scale up"*) :

- On grossit la machine
- Simple, rien à changer dans le code
- Mais il y a un plafond (et un prix)
- Toujours un *"Single Point Of Failure"*
- Souvent un redémarrage pour changer de taille

Horizontal (*"scale out"*) :

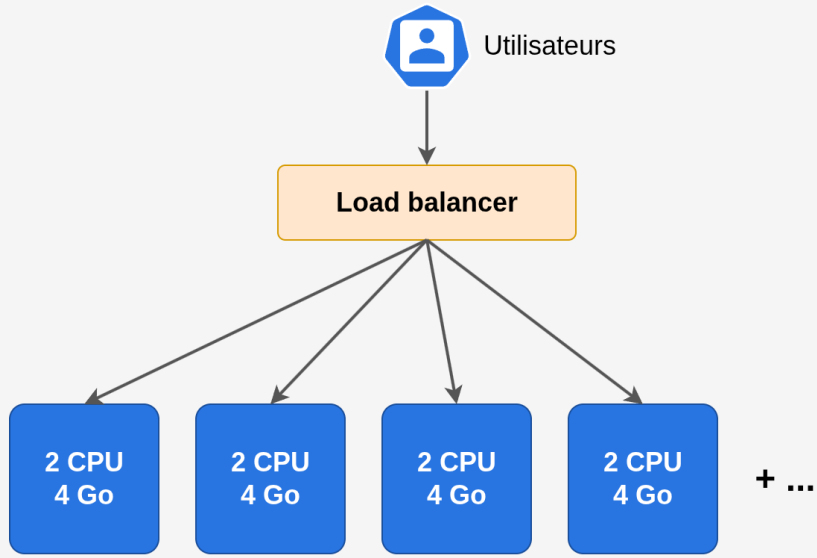
- On multiplie les instances
- Quasiment pas de plafond
- Résilient: une instance tombe, les autres prennent le relais
- Mais il faut répartir la charge et gérer toutes ces instances

Scale vertical (scale up)



Un plafond, un prix, un SPOF

Scale horizontal (scale out)



Plus d'instances, pas de plafond, résilient

Ce que l'on voit dans les logs :

```
postgres | FATAL: remaining connection slots are reserved for
           non-replication superuser connections
api      | java.sql.SQLException: Too many connections
nginx    | [error] 12345#0: *6789 upstream prematurely closed connection
nginx    | "GET /checkout HTTP/1.1" 502 Bad Gateway
api      | java.io.IOException: Too many open files
```

Ce que voient les utilisateurs :

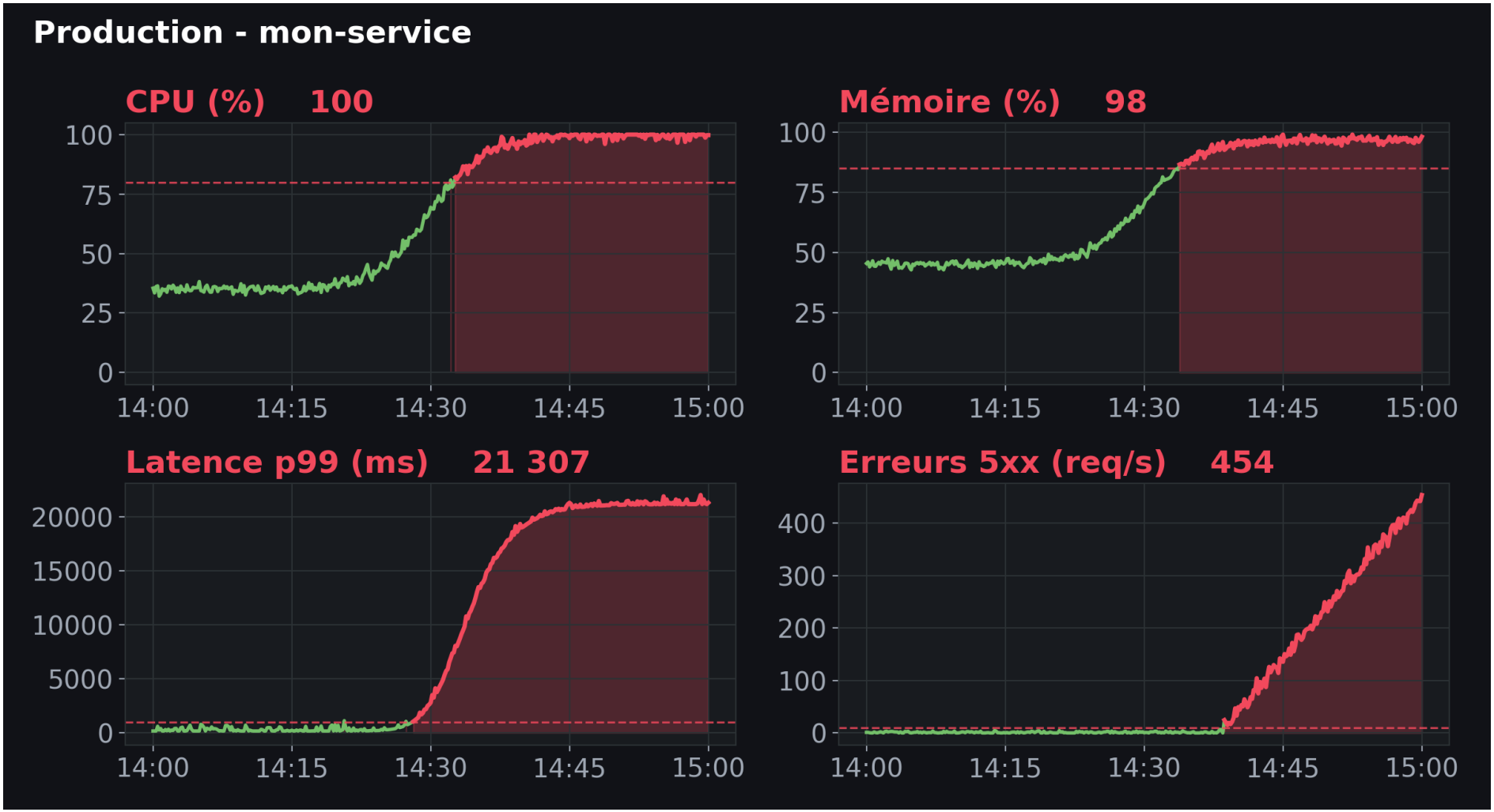
- Des pages qui mettent plus de 20 secondes à charger
- Des sessions qui expirent prématurément
- Des déconnexions aléatoires (le backend est trop lent à répondre)

Ce que voient les utilisateurs :

- Des pages qui mettent plus de 20 secondes à charger
- Des sessions qui expirent prématurément
- Des déconnexions aléatoires (le backend est trop lent à répondre)

Ce que voit le monitoring :

- CPU à 100%
- RAM saturée
- Swap utilisé à fond


Production - mon-service
CPU (%) 100
100
75
50
25
0
14:00
14:15
14:30
14:45
15:00
Mémoire (%) 98
100
75
50
25
0
14:00
14:15
14:30
14:45
15:00
Latence p99 (ms) 21 307
20000
15000
10000
5000
0
14:00
14:15
14:30
14:45
15:00
Erreurs 5xx (req/s) 454
400
300
200
100
0
14:00
14:15
14:30
14:45
15:00

Bilan :

- Une seule machine, même énorme, ne suffit plus
- Il faut plusieurs instances de notre service
- Il faut répartir le trafic entre ces instances
- Il faut en ajouter quand ça chauffe et en retirer quand c'est calme
- Idéalement ça se passe **automatiquement** $\Rightarrow$ #AutomationFirst

Kubernetes !

Un peu d'histoire:

- Depuis le milieu des années 2000, Google fait tourner toute sa production dans des conteneurs avec **Borg** (puis Omega)
- 2014: Google ouvre une version open source de ces idées: **Kubernetes**
- 2015: version 1.0 et don à la toute nouvelle **Cloud Native Computing Foundation** (CNCF)
- Aujourd'hui: le standard de facto de l'orchestration de conteneurs, proposé par tous les clouds (GKE, EKS, AKS, Kapsule, ...)

- Un **orchestrateur de conteneurs**: on lui confie des machines et des conteneurs, il décide où et comment les faire tourner
- Il les surveille, les redémarre, les met à jour et les multiplie à notre place
- Le même outil partout: sur un laptop, dans un datacenter ou chez n'importe quel cloud



Gestion du lifecycle :

- **Self-healing**: redémarrer un conteneur qui crashe
- Remplacer les conteneurs d'une machine qui tombe
- **Rolling updates**: mettre à jour les instances une par une, sans coupure
- **Rollback**: revenir en arrière quand la nouvelle version est cassée

Comment Kubernetes gère le lifecycle :

- On décrit l'état voulu (*"3 instances de mon app en v2"*), Kubernetes le maintient en permanence
- Un conteneur crashe ou ne répond plus ? Il est redémarré automatiquement
- Une machine tombe ? Ses conteneurs sont relancés ailleurs
- Nouvelle version ? Les instances sont remplacées progressivement, et seulement quand les nouvelles sont prêtes
- Un problème pendant la mise à jour ? Une commande pour revenir à la version précédente

Résilience :

- Plusieurs instances du même service
- Réparties sur plusieurs machines (voire plusieurs zones)
- Une instance tombe ? Le trafic part vers les autres
- Il faut donc du **load balancing** et des **déploiements multi-instances**

Comment Kubernetes assure la résilience :

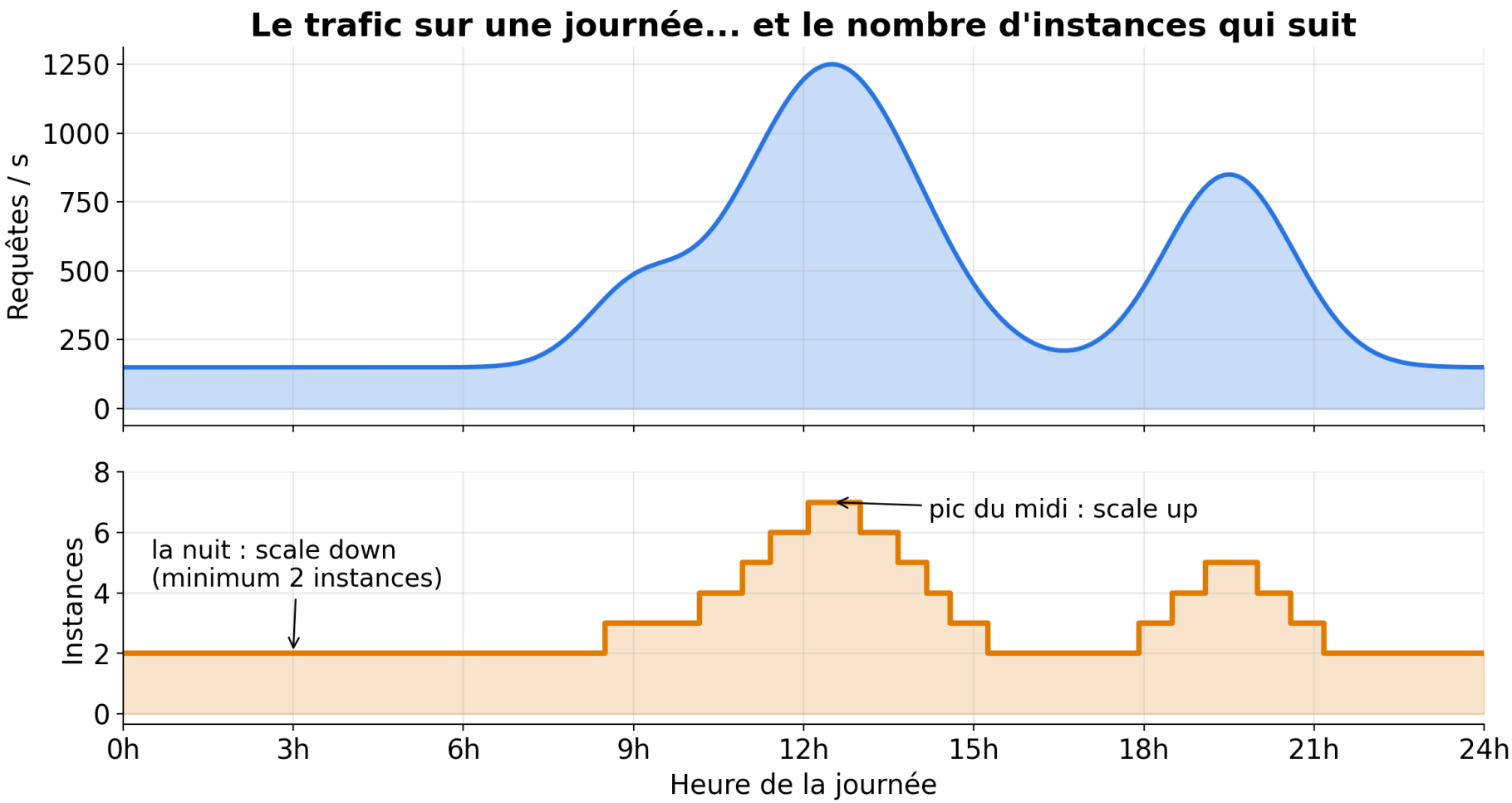
- On demande N instances: il les crée et les répartit sur plusieurs machines (et zones)
- Il fournit une adresse unique qui répartit le trafic entre toutes les instances
- Seules les instances en bonne santé reçoivent du trafic
- Une instance disparaît ? Le trafic est redirigé et une remplaçante est créée

Scalabilité :

- Un nombre d'instances **variable** selon la charge
- Ajouter des instances quand ça chauffe
- Les retirer quand c'est calme (pour ne pas payer pour rien)
- Mettre à jour le load balancing à chaque ajout/destruction

Comment Kubernetes gère la scalabilité :

- Changer le nombre d'instances = changer un chiffre, Kubernetes fait le reste
- Ou automatiquement: il ajoute et retire des instances selon la charge mesurée (CPU, mémoire, ...)
- Le load balancing suit tout seul: les nouvelles instances reçoivent du trafic dès qu'elles sont prêtes


Le trafic sur une journée... et le nombre d'instances qui suit
Requêtes / s
1250
1000
750
500
250
0
Instances
8
6
4
2
0
la nuit : scale down
(minimum 2 instances)
pic du midi : scale up
0h
3h
6h
9h
12h
15h
18h
21h
24h
Heure de la journée

Questions ?

Kubernetes, c'est:

- Un **orchestrateur de conteneurs**

Kubernetes, c'est:

- Un **orchestrateur de conteneurs**
- Une **plateforme extensible** (on peut lui apprendre de nouveaux objets)

Kubernetes, c'est:

- Un **orchestrateur de conteneurs**
- Une **plateforme extensible** (on peut lui apprendre de nouveaux objets)
- Une **abstraction entre dev et ops**: le dev décrit ce qu'il veut, la plateforme s'occupe du reste

Kubernetes, c'est:

- Un **orchestrateur de conteneurs**
- Une **plateforme extensible** (on peut lui apprendre de nouveaux objets)
- Une **abstraction entre dev et ops**: le dev décrit ce qu'il veut, la plateforme s'occupe du reste
- Une **API**: tout est une ressource que l'on crée, lit, modifie et supprime

Le principe fondamental: le **déclaratif**

- On ne dit pas *"lance 3 conteneurs"* (impératif)

Le principe fondamental: le **déclaratif**

- On ne dit pas *"lance 3 conteneurs"* (impératif)
- On dit *"je veux 3 conteneurs"* (déclaratif): c'est l'**état désiré**

Le principe fondamental: le **déclaratif**

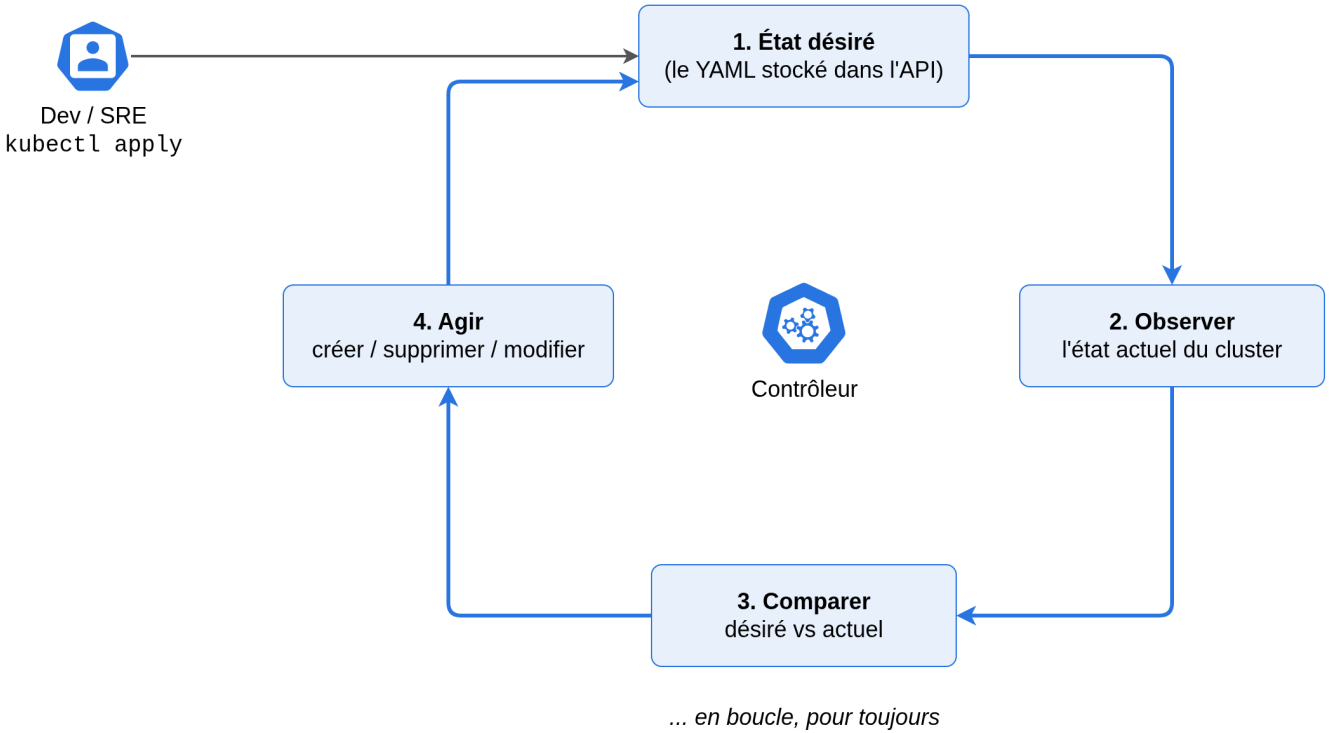
- On ne dit pas *"lance 3 conteneurs"* (impératif)
- On dit *"je veux 3 conteneurs"* (déclaratif): c'est l'**état désiré**
- Des contrôleurs comparent en permanence l'état désiré et l'**état actuel**

Le principe fondamental: le **déclaratif**

- On ne dit pas *"lance 3 conteneurs"* (impératif)
- On dit *"je veux 3 conteneurs"* (déclaratif): c'est l'**état désiré**
- Des contrôleurs comparent en permanence l'état désiré et l'**état actuel**
- Et agissent pour les réconcilier: c'est la **boucle de réconciliation**

Le principe fondamental: le **déclaratif**

- On ne dit pas *"lance 3 conteneurs"* (impératif)
- On dit *"je veux 3 conteneurs"* (déclaratif): c'est l'**état désiré**
- Des contrôleurs comparent en permanence l'état désiré et l'**état actuel**
- Et agissent pour les réconcilier: c'est la **boucle de réconciliation**
- Même idée que Terraform (Jour 1), mais en continu



Exemple

Désiré : 3 conteneurs de mon app
Actuel : 2 (un conteneur a crashé)

⇒ **le contrôleur en relance 1**

Ce que fait Kubernetes :

- Déploie des applications conteneurisées sur un grand nombre de nœuds
- Automatise les *rolling updates*
- Détecte les incidents et les répare (*self-healing*)
- Scale automatiquement en fonction de métriques sur-mesure
- Load balance le trafic entre les conteneurs

Pour orchestrer des conteneurs, Kubernetes a besoin de machines: les **nœuds**

- Un nœud = une machine (souvent une VM) qui prête son CPU et sa mémoire au cluster
- Avec des attributs: CPU, mémoire, zone géographique, ...
- Kubernetes voit tous les nœuds comme un seul grand réservoir de capacité
- Il choisit sur quel nœud placer chaque conteneur en fonction de ces attributs

Enrôler un nœud dans le cluster :

- On installe sur la machine un *container runtime* (containerd) et l'agent Kubernetes (`kubelet`)
- Le `kubelet` s'enregistre auprès du control plane (`kubeadm join ...`)
- Le nœud apparaît dans le cluster et reçoit des conteneurs
- Dans le cloud: on définit un *node pool* (taille et nombre de VMs) et le fournisseur s'occupe du reste

Les nœuds d'un cluster kind (Kubernetes dans Docker) :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get nodes
NAME                 STATUS   ROLES           AGE   VERSION
kind-control-plane   Ready    control-plane   12m   v1.33.1
kind-worker          Ready    <none>          12m   v1.33.1
kind-worker2         Ready    <none>          12m   v1.33.1
```

Un cluster, deux types de nœuds :

Control plane (le cerveau) :

- `kube-apiserver`: la porte d'entrée, l'API
- `etcd`: la base clé/valeur qui stocke l'état du cluster
- `kube-scheduler`: choisit le nœud de chaque pod
- `kube-controller-manager`: les boucles de réconciliation

Workers (les bras) :

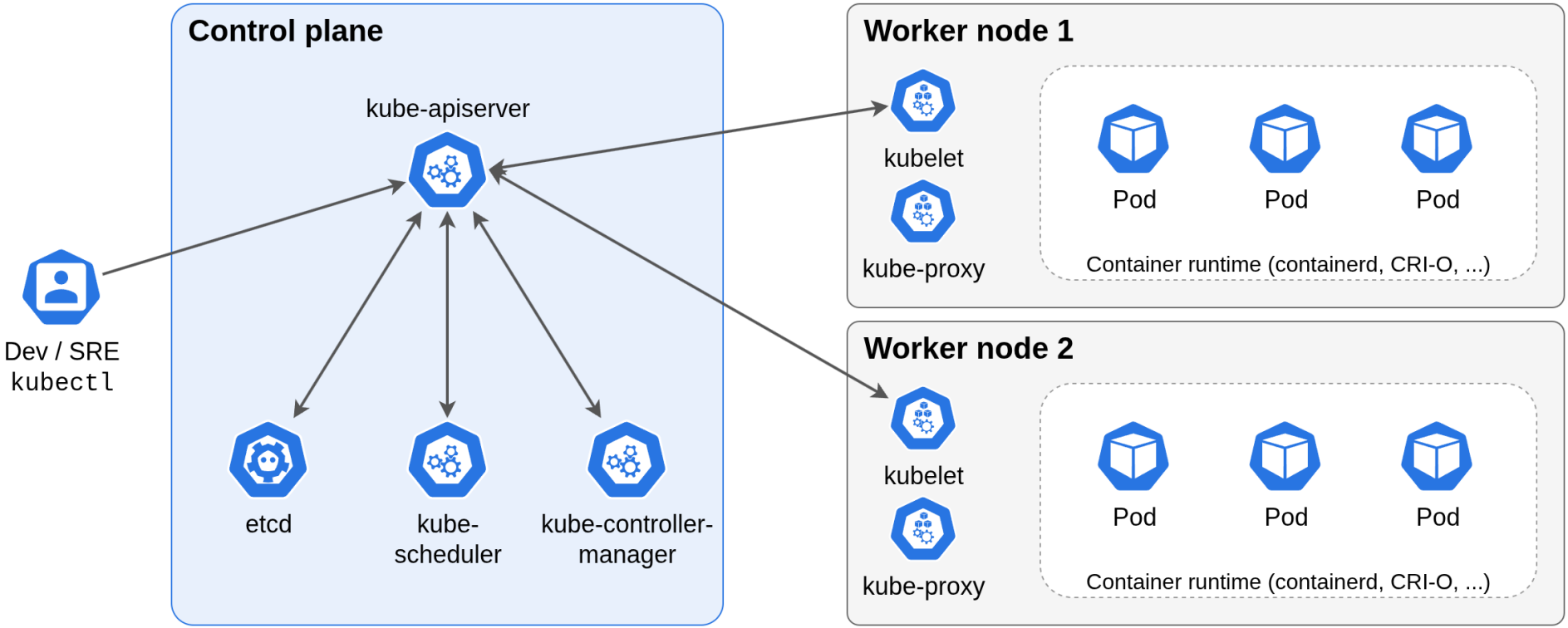
Un cluster, deux types de nœuds :

Control plane (le cerveau) :

- `kube-apiserver`: la porte d'entrée, l'API
- `etcd`: la base clé/valeur qui stocke l'état du cluster
- `kube-scheduler`: choisit le nœud de chaque pod
- `kube-controller-manager`: les boucles de réconciliation

Workers (les bras) :

- `kubelet`: l'agent qui lance les conteneurs sur le nœud
- `kube-proxy`: le réseau des Services
- *Container runtime*: containerd, CRI-O, ...


Control plane
kube-apiserver
etcd
kube-scheduler
kube-controller-manager
Dev / SRE
kubectl
Worker node 1
kubelet
kube-proxy
Pod
Pod
Pod
Container runtime (containerd, CRI-O, ...)
Worker node 2
kubelet
kube-proxy
Pod
Pod
Pod
Container runtime (containerd, CRI-O, ...)

Le control plane... ce sont aussi des pods !

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get pods -n kube-system
NAME                                         READY   STATUS    RESTARTS   AGE
coredns-674b8bbfcf-8qv2x                     1/1     Running   0          12m
coredns-674b8bbfcf-wz7lm                     1/1     Running   0          12m
etcd-kind-control-plane                      1/1     Running   0          12m
kindnet-4kx9d                                1/1     Running   0          12m
kindnet-q8m2z                                1/1     Running   0          12m
kindnet-zt7rw                                1/1     Running   0          12m
kube-apiserver-kind-control-plane            1/1     Running   0          12m
kube-controller-manager-kind-control-plane   1/1     Running   0          12m
kube-proxy-6hn2c                             1/1     Running   0          12m
kube-proxy-9tj4w                             1/1     Running   0          12m
kube-proxy-xl5pd                             1/1     Running   0          12m
kube-scheduler-kind-control-plane            1/1     Running   0          12m
```

Deux mondes à ne pas confondre :

Les composants (le moteur) :

- Les programmes qui font tourner Kubernetes: `kube-apiserver`, `etcd`, `scheduler`, `kubelet`, ...
- Installés et opérés par l'équipe plateforme (ou le cloud)
- L'utilisateur ne les touche pas

Les ressources (le volant) :

- Les objets que l'on décrit en YAML et envoie à l'API: Pod, Deployment, Service, ...
- Manipulées par les devs et les ops au quotidien
- C'est ce que l'on voit dans la suite

Les ressources Kubernetes :

- Décrites en YAML (le fameux *"YAML engineering"*)
- Envoyées à l'API, par exemple avec
  `kubectl apply -f my-app.yaml`
- Toujours la même structure: `apiVersion`, `kind`, `metadata`, `spec`
- Rangées dans des **namespaces** et identifiées par des **labels**

Les **namespaces**: ranger le cluster

- Un dossier logique qui regroupe des ressources: une équipe, une application, un environnement, ...
- Les noms sont uniques par namespace: deux équipes peuvent avoir chacune leur `my-service`
- Les droits (RBAC) et les règles réseau (NetworkPolicies) s'appliquent par namespace
- Ce n'est pas une frontière de sécurité à lui seul: il faut le combiner avec RBAC et NetworkPolicies
- Déjà présents: `default`, `kube-system` (les composants), ...

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
```

```
# Travailler dans un namespace
kubectl get pods -n my-namespace
```

Les namespaces d'un cluster :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get namespaces
NAME                 STATUS   AGE
backend              Active   3d
data                 Active   3d
default              Active   12d
frontend             Active   3d
kube-node-lease      Active   12d
kube-public          Active   12d
kube-system          Active   12d
my-namespace         Active   2m
```

Les **Pods**: l'unité de base pour faire tourner des conteneurs

- Un ou plusieurs conteneurs qui partagent le réseau (même IP) et des volumes
- Le scheduler place chaque pod sur un nœud
- Éphémère: un pod qui meurt n'est pas ressuscité, il est remplacé
- On ne crée (presque) jamais un pod à la main

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
  labels:
    foo: bar
spec:
  containers:
    - name: my-container
      image: myapp:v1.0.0
      command: ['/bin/my-app']
      args: ['--migrate-db', '--db-host=12.34.56.78']
```

En vrai :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl apply -f my-pod.yaml
pod/my-pod created
$ kubectl get pods -n my-namespace
NAME     READY   STATUS    RESTARTS   AGE
my-pod   1/1     Running   0          8s
$ kubectl describe pod my-pod -n my-namespace | grep -E '^(Node|Labels|IP):'
Node:         kind-worker/172.18.0.3
Labels:       foo=bar
IP:           10.244.1.5
```

Les **ReplicaSets**: des pods identiques

- Garantit qu'un nombre donné de pods identiques (`replicas`) tourne en permanence
- Retrouve ses pods grâce aux labels (`selector`)
- Crée les nouveaux pods à partir d'un modèle (`template`)
- Un pod meurt ? Le ReplicaSet en recrée un $\Rightarrow$ *self-healing*

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      foo: bar
  template:
    metadata:
      labels:
        foo: bar
    spec:
      containers:
        - name: my-container
          image: myapp:v1.0.0
          ports:
            - containerPort: 3000
```

Le *self-healing* en direct :

elixir@sandbox: ~ (kind-sandbox)

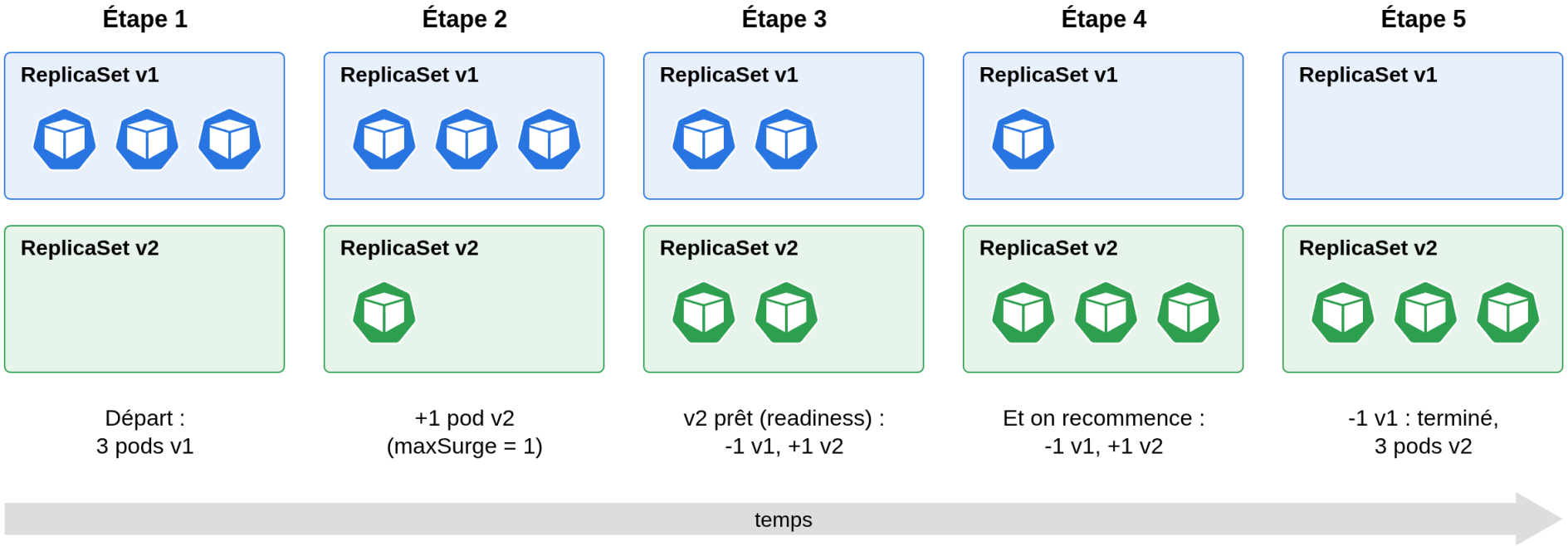
```
$ kubectl get replicaset
NAME            DESIRED   CURRENT   READY   AGE
my-replicaset   3         3         3       2m
$ kubectl get pods
NAME                  READY   STATUS    RESTARTS   AGE
my-replicaset-4xw9k   1/1     Running   0          2m
my-replicaset-bn7tq   1/1     Running   0          2m
my-replicaset-q2c8d   1/1     Running   0          2m
$ kubectl delete pod my-replicaset-bn7tq
pod "my-replicaset-bn7tq" deleted
$ kubectl get pods
NAME                  READY   STATUS    RESTARTS   AGE
my-replicaset-4xw9k   1/1     Running   0          2m
my-replicaset-q2c8d   1/1     Running   0          2m
my-replicaset-zk5vh   1/1     Running   0          4s
```

Les **Deployments**: zéro downtime

- Gère des ReplicaSets à notre place
- Nouvelle version = nouveau ReplicaSet, l'ancien est vidé progressivement (*rolling update*)
- Rythme contrôlé avec `maxSurge` et `maxUnavailable`
- Garde un historique: `kubectl rollout undo` pour revenir en arrière
- C'est l'objet que l'on utilise au quotidien pour une application *stateless*

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      foo: bar
  template:
    metadata:
      labels:
        foo: bar
    spec:
      containers:
        - name: my-container
          image: myapp:v1.0.0
          ports:
            - containerPort: 3000
```

Deployment `my-deployment` : image v1 ⇒ v2



Un rolling update en direct :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl set image deployment/my-deployment my-container=myapp:v1.1.0
deployment.apps/my-deployment image updated
$ kubectl rollout status deployment/my-deployment
Waiting for deployment "my-deployment" rollout to finish: 1 out of 3 new replicas have
been updated...
Waiting for deployment "my-deployment" rollout to finish: 2 out of 3 new replicas have
been updated...
Waiting for deployment "my-deployment" rollout to finish: 1 old replicas are pending
termination...
deployment "my-deployment" successfully rolled out
$ kubectl get replicaset
NAME                       DESIRED   CURRENT   READY   AGE
my-deployment-5f6b9c7d8    3         3         3       40s
my-deployment-7c9d8b6f4d   0         0         0       3d
$ kubectl rollout undo deployment/my-deployment
deployment.apps/my-deployment rolled back
```

Et les applications avec un état (bases de données, Kafka, ...) ?

- Un **StatefulSet** ressemble à un Deployment, mais :
- Des noms stables et ordonnés: `my-db-0`, `my-db-1`, `my-db-2`
- Un disque dédié par pod qui le suit s'il est recréé (on verra le stockage plus loin)
- Démarrage et arrêt dans l'ordre

En pratique: préférer un operator (CloudNativePG, ...) ou un service managé, qui gère aussi la réplication et le failover

Des noms stables et un disque par pod :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get statefulset
NAME    READY   AGE
my-db   3/3     5m
$ kubectl get pods -l app=my-db
NAME      READY   STATUS    RESTARTS   AGE
my-db-0   1/1     Running   0          5m
my-db-1   1/1     Running   0          4m
my-db-2   1/1     Running   0          4m
$ kubectl get pvc -o custom-
  columns=NAME:.metadata.name,STATUS:.status.phase,SIZE:.status.capacity.storage
NAME           STATUS   SIZE
data-my-db-0   Bound    10Gi
data-my-db-1   Bound    10Gi
data-my-db-2   Bound    10Gi
```

Les **DaemonSets**: un pod sur chaque nœud

- Pas de `replicas`: autant de pods que de nœuds
- Un nouveau nœud arrive ? Il reçoit automatiquement son pod
- Pour les agents d'infrastructure: collecte de logs, monitoring, réseau (`kube-proxy`, CNI), ...

Un pod par nœud :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get daemonset kindnet -n kube-system
NAME      DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
kindnet   3         3         3       3            3           <none>          12d
$ kubectl get pods -n kube-system -l app=kindnet -o custom-
  columns=NAME:.metadata.name,NODE:.spec.nodeName
NAME            NODE
kindnet-4kx9d   kind-control-plane
kindnet-q8m2z   kind-worker
kindnet-zt7rw   kind-worker2
```

Les **Jobs** et **CronJobs**: des tâches qui se terminent

- **Job**: lance un pod qui s'exécute jusqu'au bout puis s'arrête (migration de base de données, import, ...)
- Relance en cas d'échec, jusqu'à un nombre d'essais (`backoffLimit`)
- **CronJob**: crée un Job selon un planning au format cron (sauvegarde toutes les nuits, ...)

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 3 * * *"
  jobTemplate:
    spec:
      backoffLimit: 3
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: my-backup:v1.0.0
```

Un CronJob, les Jobs qu'il crée et leurs pods :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get cronjob
NAME             SCHEDULE    TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
nightly-backup   0 3 * * *   <none>     False     0        6h              3d
$ kubectl get jobs
NAME                       STATUS     COMPLETIONS   DURATION   AGE
nightly-backup-29355780    Complete   1/1           42s        2d6h
nightly-backup-29357220    Complete   1/1           39s        30h
nightly-backup-29358660    Complete   1/1           44s        6h
$ kubectl get pods
NAME                             READY   STATUS      RESTARTS   AGE
nightly-backup-29358660-q7x4m    0/1     Completed   0          6h
```

Les **HorizontalPodAutoscalers**: scaler en fonction de la charge

- Ajuste automatiquement le nombre de `replicas` d'un Deployment
- En fonction de métriques: CPU, mémoire, ou métriques sur-mesure (requêtes/s, taille d'une file, ...)
- Entre un minimum et un maximum de replicas
- Nécessite un `metrics-server` dans le cluster
- Le CPU est mesuré par rapport aux `requests` des conteneurs: pas de requests, pas d'HPA !

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-deployment
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

L'HPA en action pendant un pic de charge :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get hpa my-hpa --watch
NAME     REFERENCE                   TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
my-hpa   Deployment/my-deployment    cpu: 12%/60%    1         10        1          5m
my-hpa   Deployment/my-deployment    cpu: 187%/60%   1         10        1          6m
my-hpa   Deployment/my-deployment    cpu: 187%/60%   1         10        4          6m
my-hpa   Deployment/my-deployment    cpu: 95%/60%    1         10        8          7m
my-hpa   Deployment/my-deployment    cpu: 58%/60%    1         10        8          8m
```

Les **Services**: du load balancing entre les pods

- Les pods sont éphémères et changent d'IP: impossible de les viser directement
- Un Service offre une IP et un nom DNS stables (`my-service.my-namespace.svc`)
- Il répartit le trafic entre tous les pods qui portent les mêmes labels
- Plusieurs types: `ClusterIP` (interne), `NodePort`, `LoadBalancer` (externe via le cloud)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    foo: bar
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

Le Service et les IPs des pods derrière lui :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get service my-service
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
my-service   ClusterIP   10.96.142.27   <none>        80/TCP    3m
$ kubectl get endpointslices -l kubernetes.io/service-name=my-service
NAME               ADDRESSTYPE   PORTS   ENDPOINTS                         AGE
my-service-x7k2p   IPv4          8080    10.244.1.7,10.244.2.4,10.244.2.5   3m
$ kubectl get pods -l foo=bar -o custom-columns=NAME:.metadata.name,IP:.status.podIP
NAME                             IP
my-deployment-7c9d8b6f4d-2xk8p   10.244.1.7
my-deployment-7c9d8b6f4d-9rjvn   10.244.2.4
my-deployment-7c9d8b6f4d-tq4wl   10.244.2.5
```

Les **Ingresses**: router les requêtes qui viennent de l'extérieur

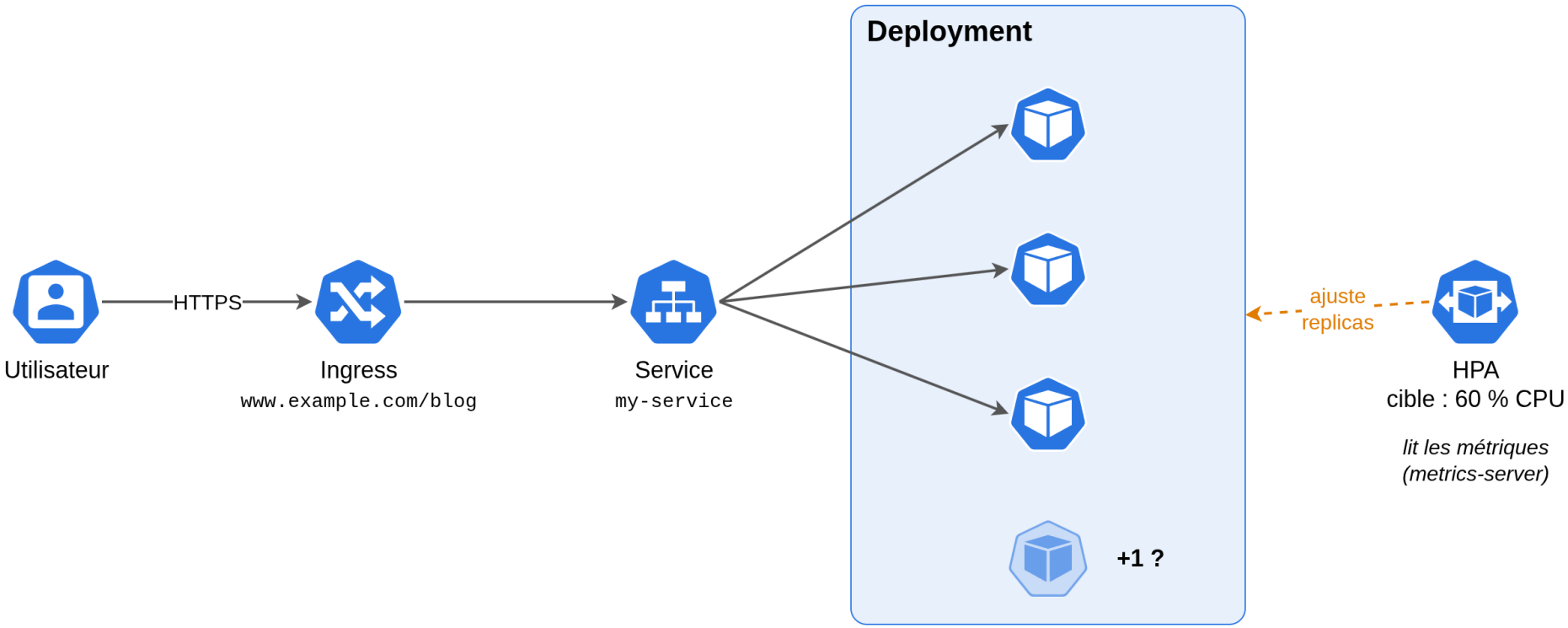
- Routage HTTP(S) par nom de domaine et par chemin vers des Services
- Un seul point d'entrée pour plusieurs applications
- Gère la terminaison TLS (HTTPS)
- Nécessite un *Ingress controller* (ingress-nginx, Traefik, ...)
- L'API Ingress est gelée: son successeur est la **Gateway API**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
    - host: www.example.com
      http:
        paths:
          - path: /blog
            pathType: Prefix
            backend:
              service:
                name: my-service
                port:
                  number: 80
```

En vrai :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get ingress
NAME         CLASS   HOSTS             ADDRESS     PORTS   AGE
my-ingress   nginx   www.example.com   localhost   80      2m
$ curl -s http://www.example.com/blog --resolve www.example.com:80:127.0.0.1
<h1>Bienvenue sur le blog !</h1>
```


Deployment
Utilisateur
HTTPS
Ingress
www.example.com/blog
Service
my-service
ajuste
replicas
HPA
cible : 60 % CPU
lit les métriques
(metrics-server)
+1 ?

Les **ConfigMaps** et les **Secrets**: la configuration hors de l'image

- La même image du dev à la prod, seule la configuration change
- **ConfigMap**: de la configuration non sensible (URL, niveau de logs, fichier de config, ...)
- **Secret**: des données sensibles (mots de passe, tokens, certificats, ...)
- Injectés dans les conteneurs en variables d'environnement ou en fichiers
- Attention: un Secret est encodé en base64, pas chiffré !

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  LOG_LEVEL: info
  DB_HOST: my-db
---
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
stringData:
  DB_PASSWORD: s3cr3t
```

```yaml
# Dans le Deployment
containers:
  - name: my-container
    image: myapp:v1.0.0
    envFrom:
      - configMapRef:
          name: my-config
    env:
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef:
            name: my-secret
            key: DB_PASSWORD
```

Un Secret n'est pas chiffré :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get secret my-secret -o yaml
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  namespace: my-namespace
type: Opaque
data:
  DB_PASSWORD: czNjcjN0
$ echo czNjcjN0 | base64 -d
s3cr3t
```

Ranger ses fichiers: **Kustomize** (intégré à `kubectl`)

```
my-app/
`-- base/
    |-- kustomization.yaml
    |-- namespace.yaml
    |-- configmap.yaml
    |-- deployment.yaml
    |-- service.yaml
    |-- ingress.yaml
    `-- hpa.yaml
```

`base/kustomization.yaml` :

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: my-app
resources:
  - namespace.yaml
  - configmap.yaml
  - deployment.yaml
  - service.yaml
  - ingress.yaml
  - hpa.yaml
```

```
kubectl apply -k base/
```

Une base, un *overlay* par environnement :

```
my-app/
|-- base/
|   |-- kustomization.yaml
|   |-- namespace.yaml
|   |-- configmap.yaml
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- ingress.yaml
|   `-- hpa.yaml
`-- overlays/
    |-- test/
    |   |-- kustomization.yaml
    |   `-- configmap-patch.yaml
    `-- prod/
        |-- kustomization.yaml
        |-- deployment-patch.yaml
        `-- hpa-patch.yaml
```

`overlays/prod/kustomization.yaml` :

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: my-app-prod
resources:
  - ../../base
images:
  - name: myapp
    newTag: v1.2.0
patches:
  - path: deployment-patch.yaml
  - path: hpa-patch.yaml
```

```
kubectl apply -k overlays/prod/
```

On ne copie pas les YAML: on décrit seulement les différences

Un patch en détail: on n'écrit que ce qui change

`overlays/prod/deployment-patch.yaml` :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment    # la cible
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: my-container
          resources:
            requests:
              cpu: 500m
              memory: 512Mi
            limits:
              memory: 1Gi
```

Résultat:

`kubectl kustomize overlays/prod/`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  namespace: my-app-prod  # overlay
spec:
  replicas: 3             # patch
  template:
    spec:
      containers:
        - name: my-container
          image: myapp:v1.2.0  # overlay
          ports:
            - containerPort: 3000  # base
          resources:               # patch
            requests:
              cpu: 500m
              memory: 512Mi
            limits:
              memory: 1Gi
```

Questions ?

Kubernetes ne fait pas tout tout seul :

- Il fait ce qu'on lui demande... et seulement ce qu'on lui demande
- Une application mal décrite sera mal orchestrée
- Il **délègue**: il orchestre, mais s'appuie sur l'application et sur des plugins pour le reste

Comment Kubernetes délègue :

À l'application, via un contrat décrit dans le YAML :

- *"Est-ce que je vais bien ?"* → des endpoints de santé (**probes**)
- *"De quoi j'ai besoin ?"* → **requests** / **limits**
- *"Avec quels droits je tourne ?"* → `securityContext`, RBAC
- *"Comment je me configure ?"* → ConfigMaps, Secrets

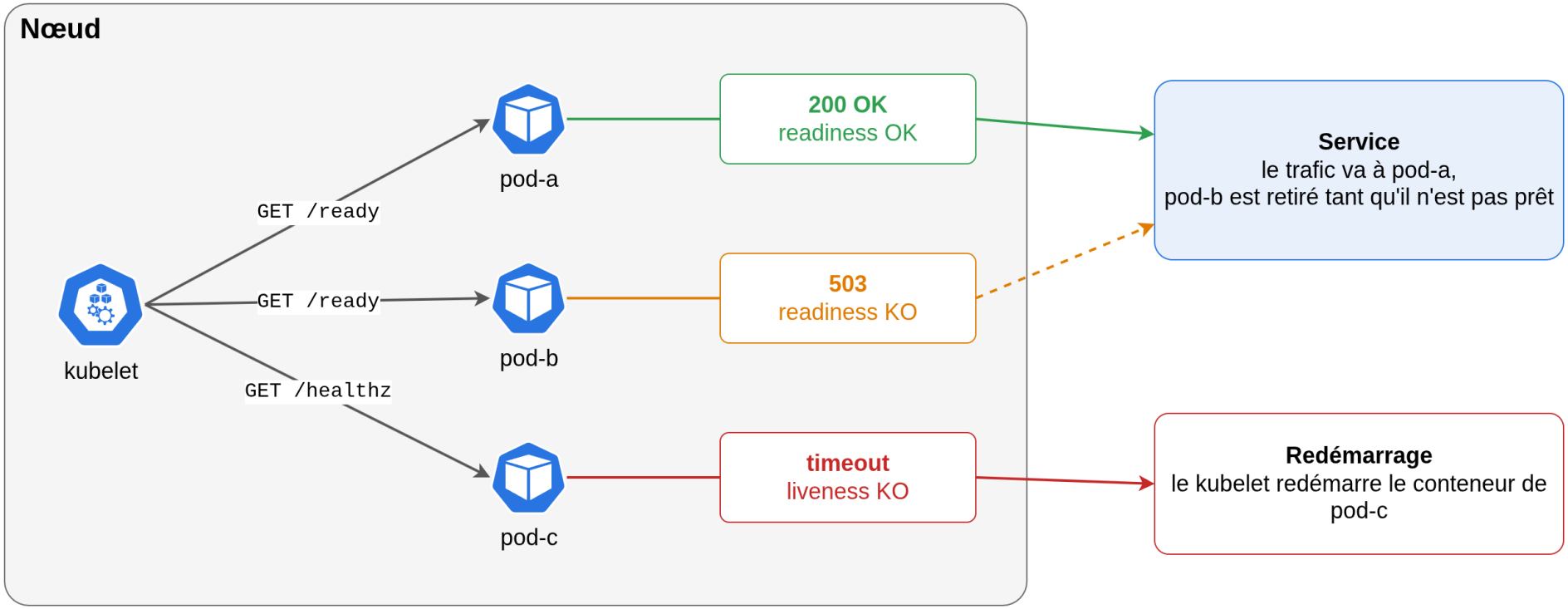
À des plugins, via des interfaces standard :

- Le réseau des pods → un plugin **CNI**
- Le stockage → un driver **CSI**
- Le trafic entrant → un *Ingress controller*
- Les métriques de l'HPA → `metrics-server`

Les bonnes pratiques, c'est bien remplir ce contrat sur quatre axes: **disponibilité**, **sécurité**, **scalabilité**, **maintenabilité**

Une **probe**, c'est quoi ?

- Un test de santé que le `kubelet` exécute régulièrement sur chaque conteneur
- Trois façons de tester: une requête HTTP (`GET /healthz` doit répondre 200), une connexion TCP, ou une commande dans le conteneur
- Si le test échoue plusieurs fois de suite (`failureThreshold`), Kubernetes réagit
- C'est à l'application d'exposer ces endpoints: c'est elle qui sait si elle va bien


Nœud
kubelet
GET /ready
GET /ready
GET /healthz
pod-a
pod-b
pod-c
200 OK
readiness OK
503
readiness KO
timeout
liveness KO
Service
le trafic va à pod-a,
pod-b est retiré tant qu'il n'est pas prêt
Redémarrage
le kubelet redémarre le conteneur de
pod-c

Les trois probes :

- `livenessProbe`: mon conteneur est-il vivant ? Sinon, Kubernetes le redémarre
- `readinessProbe`: mon conteneur est-il prêt à recevoir du trafic ? Sinon, le Service ne lui envoie rien
- `startupProbe`: laisser le temps à une application lente de démarrer avant de tester les deux autres
- Le *self-healing* commence ici: sans probes, Kubernetes ne sait pas que votre application est cassée
- Attention: une liveness qui dépend de la base de données = tous les pods redémarrent en même temps quand la base tousse

```yaml
containers:
  - name: my-container
    image: myapp:v1.0.0
    livenessProbe:
      httpGet:
        path: /healthz
        port: 3000
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /ready
        port: 3000
      periodSeconds: 5
```

Une readiness probe qui échoue: le pod ne reçoit plus de trafic

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get pods
NAME                             READY   STATUS    RESTARTS   AGE
my-deployment-7c9d8b6f4d-2xk8p   1/1     Running   0          10m
my-deployment-7c9d8b6f4d-9rjvn   0/1     Running   0          10m
my-deployment-7c9d8b6f4d-tq4wl   1/1     Running   0          10m
$ kubectl describe pod my-deployment-7c9d8b6f4d-9rjvn
[...]
Events:
  Type     Reason     Age                  From     Message
  ----     ------     ----                 ----     -------
  Warning  Unhealthy  12s (x31 over 5m)  kubelet  Readiness probe failed: HTTP probe
failed with statuscode: 503
```

Définir des **requests** / **limits**:

- `requests`: ce dont le conteneur a besoin, utilisé par le scheduler pour choisir un nœud ⇒ le *"SLA de scheduling"*
- `limits`: le maximum autorisé, les garde-fous
- Dépasser la limite CPU = ralenti (*throttling*)
- Dépasser la limite mémoire = tué (`OOMKilled`)
- Des limits trop serrées ⇒ OOMKill en boucle

```yaml
containers:
  - name: my-container
    image: myapp:v1.0.0
    resources:
      requests:
        cpu: 250m
        memory: 256Mi
      limits:
        memory: 512Mi
```

Une limite mémoire trop serrée :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get pods
NAME                              READY   STATUS             RESTARTS      AGE
my-deployment-7c9d8b6f4d-2xk8p    1/1     Running            0             10m
my-deployment-7c9d8b6f4d-tq4wl    0/1     CrashLoopBackOff   4 (21s ago)   3m
$ kubectl describe pod my-deployment-7c9d8b6f4d-tq4wl
[...]
    Last State:     Terminated
      Reason:       OOMKilled
      Exit Code:    137
    Restart Count:  4
    Limits:
      memory:  512Mi
    Requests:
      cpu:     250m
      memory:  256Mi
```

Des requests trop grosses: le scheduler ne trouve aucun nœud

elixir@sandbox: ~ (kind-sandbox)

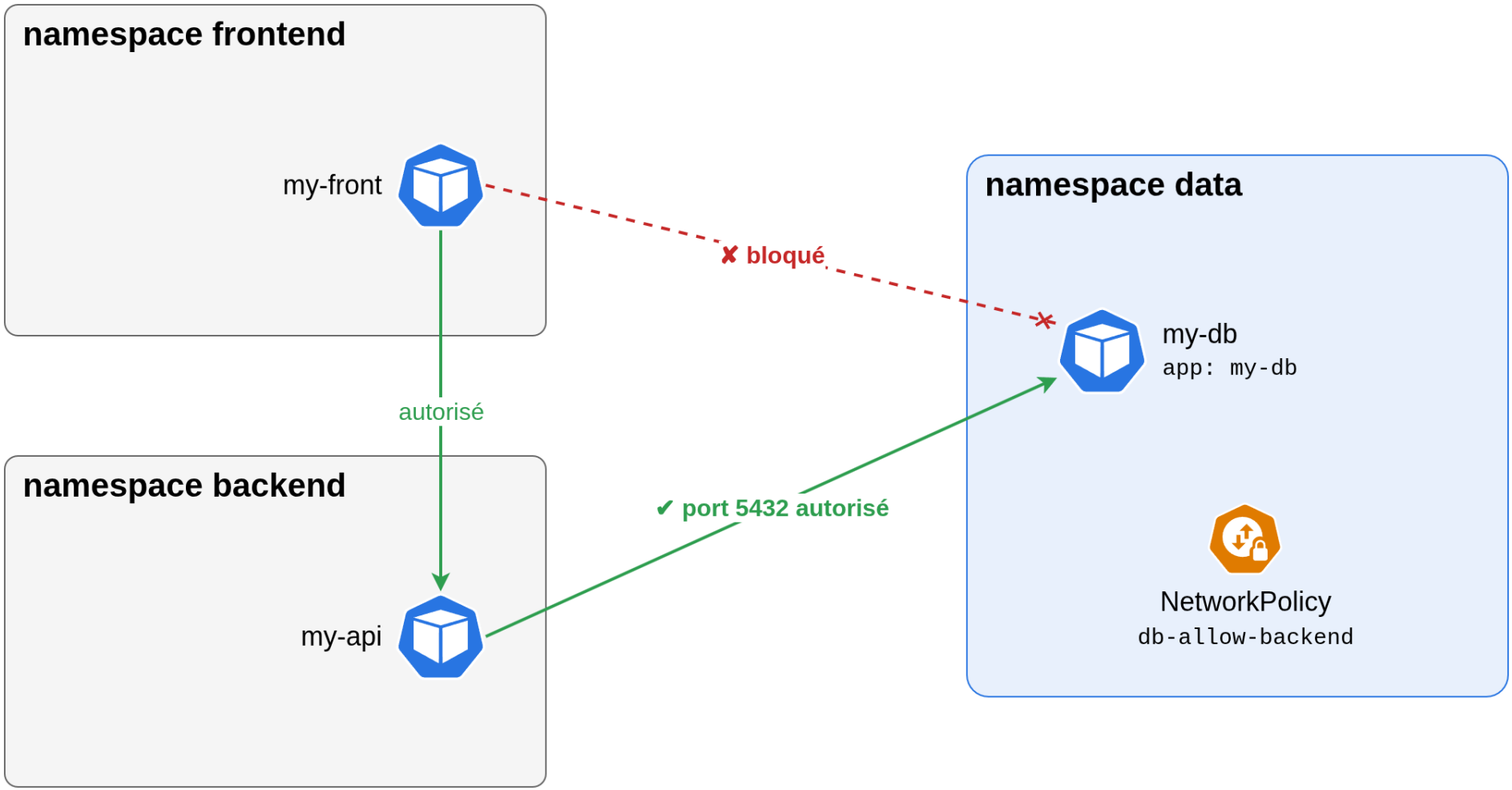
```
$ kubectl get pods
NAME                             READY   STATUS    RESTARTS   AGE
my-deployment-7c9d8b6f4d-2xk8p   1/1     Running   0          10m
my-deployment-7c9d8b6f4d-h8x2m   0/1     Pending   0          45s
my-deployment-7c9d8b6f4d-p4c7n   0/1     Pending   0          45s
$ kubectl describe pod my-deployment-7c9d8b6f4d-h8x2m
[...]
Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  45s   default-scheduler  0/3 nodes are available: 1
node(s) had untolerated taint {node-role.kubernetes.io/control-plane: }, 2 Insufficient
cpu.
```

Pas de mots de passe en clair :

- Jamais de secret dans l'image ni dans le YAML commité
- Les **Secrets** Kubernetes (vus plus haut) sont seulement encodés en base64
- Limiter qui peut les lire avec RBAC (voir plus loin)
- En vrai: un coffre-fort externe (Vault, cloud secret manager) synchronisé avec **External Secrets**

Isoler le trafic: les **NetworkPolicies**

- Par défaut, tous les pods peuvent parler à tous les pods, tous namespaces confondus !
- Une NetworkPolicy sélectionne des pods (par labels) et liste le trafic autorisé en entrée (`ingress`) et/ou en sortie (`egress`)
- Dès qu'un pod est sélectionné, tout ce qui n'est pas autorisé est bloqué
- Bonne pratique: tout interdire par défaut, puis ouvrir au cas par cas


namespace frontend
my-front
✘ bloqué
namespace data
my-db
app: my-db
autorisé
namespace backend
✔ port 5432 autorisé
my-api
NetworkPolicy
db-allow-backend

Seul le backend peut joindre la base de données :

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-backend
  namespace: data
spec:
  podSelector:
    matchLabels:
      app: my-db
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: backend
      ports:
        - port: 5432
```

Le backend passe, le frontend est bloqué :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get networkpolicy -n data
NAME               POD-SELECTOR   AGE
db-allow-backend   app=my-db      1m
$ kubectl exec -n backend deploy/my-api -- nc -zv -w 3 my-db.data.svc 5432
my-db.data.svc (10.96.81.12:5432) open
$ kubectl exec -n frontend deploy/my-front -- nc -zv -w 3 my-db.data.svc 5432
nc: my-db.data.svc (10.96.81.12:5432): Operation timed out
command terminated with exit code 1
```

Moindre privilège pour les conteneurs:

- Pas de conteneur qui tourne en `root`
- Système de fichiers en lecture seule
- Retirer toutes les *capabilities* Linux inutiles
- #Shiftleft: la sécurité se décrit dans le YAML, dès le début

```yaml
containers:
  - name: my-container
    image: myapp:v1.0.0
    securityContext:
      runAsNonRoot: true
      runAsUser: 10001
      readOnlyRootFilesystem: true
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
```

Qui a le droit de faire quoi ? Le **RBAC** (*Role-Based Access Control*)

- Chaque appel à l'API est authentifié (qui ?) puis autorisé (a-t-il le droit ?)
- **Role**: une liste de permissions dans un namespace: des verbes (`get`, `list`, `create`, `delete`, ...) sur des ressources
- **ClusterRole**: la même chose pour tout le cluster
- **RoleBinding** / **ClusterRoleBinding**: donne un rôle à un utilisateur, un groupe ou un **ServiceAccount**
- **ServiceAccount**: l'identité d'un pod quand il parle à l'API (Argo CD, un operator, ...)
- Tout est interdit par défaut: on ne donne que le strict nécessaire

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: my-namespace
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-pod-reader
  namespace: my-namespace
subjects:
  - kind: User
    name: alice
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Alice peut lire les pods... et rien d'autre :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl auth can-i list pods -n my-namespace --as alice
yes
$ kubectl auth can-i delete pods -n my-namespace --as alice
no
$ kubectl auth can-i get secrets -n my-namespace --as alice
no
$ kubectl get pods -n kube-system --as alice
Error from server (Forbidden): pods is forbidden: User "alice" cannot list resource
"pods" in API group "" in the namespace "kube-system"
```

Activer des HPA :

- Un HPA par application exposée à une charge variable
- Sans `requests`, les métriques ne reflètent pas la réalité
- Le CPU n'est pas tout: I/O, connexions, base de données (vous vous souvenez de `Too many connections` ?)

Déployer des applications **stateless** :

- Aucune donnée importante stockée dans le conteneur (sessions, fichiers uploadés, ...)
- L'état part dans une base de données, un cache, un stockage objet
- N'importe quel pod peut traiter n'importe quelle requête
- On peut alors ajouter et supprimer des pods sans risque

Séparer code et configuration :

- La même image du dev à la prod, seule la configuration change
- La configuration vit dans des **ConfigMaps** et des **Secrets** (vus plus haut)
- Injectée en variables d'environnement ou en fichiers

Exposer des métriques :

- Un endpoint `/metrics` au format Prometheus
- Les métriques alimentent les dashboards, les alertes... et les HPA
- Les SLO (latence, erreurs, saturation) pilotent la stratégie de scaling
- #Observability: la suite au prochain épisode (Jour 4)

Questions ?

Kubernetes est une **plateforme extensible** :

- Tout est une ressource de l'API, gérée par une boucle de réconciliation

Kubernetes est une **plateforme extensible** :

- Tout est une ressource de l'API, gérée par une boucle de réconciliation
- Et si on pouvait ajouter nos propres ressources ? $\Rightarrow$ les **CRDs**

Kubernetes est une **plateforme extensible** :

- Tout est une ressource de l'API, gérée par une boucle de réconciliation
- Et si on pouvait ajouter nos propres ressources ? $\Rightarrow$ les **CRDs**
- Et nos propres boucles de réconciliation ? $\Rightarrow$ les **operators**

Kubernetes est une **plateforme extensible** :

- Tout est une ressource de l'API, gérée par une boucle de réconciliation
- Et si on pouvait ajouter nos propres ressources ? $\Rightarrow$ les **CRDs**
- Et nos propres boucles de réconciliation ? $\Rightarrow$ les **operators**
- Et si le réseau et le stockage étaient aussi des plugins ? $\Rightarrow$ **CNI**, **CSI**

On en a déjà croisé sans le savoir :

- `kind: Application`: Argo CD
- `kind: ExternalSecret`: External Secrets
- `kind: ServiceMonitor`: Prometheus Operator
- `cilium` et `osc-bsu-csi-driver` dans le `helm list`: un CNI et un CSI (on verra Longhorn)

Les **CustomResourceDefinitions** (CRDs):

- Ajoutent un nouveau type d'objet (`kind`) à l'API Kubernetes
- Avec un schéma (OpenAPI) pour valider les champs
- Utilisables comme n'importe quelle ressource: `kubectl get backups`, RBAC, GitOps, ...
- Mais une CRD seule ne fait rien: ce n'est que de la donnée stockée dans etcd

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.example.com
spec:
  group: example.com
  scope: Namespaced
  names:
    kind: Backup
    plural: backups
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                schedule: {type: string}
                database: {type: string}
```

Une fois la CRD installée, on peut créer des objets `Backup` :

```yaml
apiVersion: example.com/v1
kind: Backup
metadata:
  name: nightly
spec:
  schedule: "0 3 * * *"
  database: my-db
```

Une fois la CRD installée, on peut créer des objets `Backup` :

```yaml
apiVersion: example.com/v1
kind: Backup
metadata:
  name: nightly
spec:
  schedule: "0 3 * * *"
  database: my-db
```

... mais qui va réellement faire la sauvegarde à 3h du matin ?

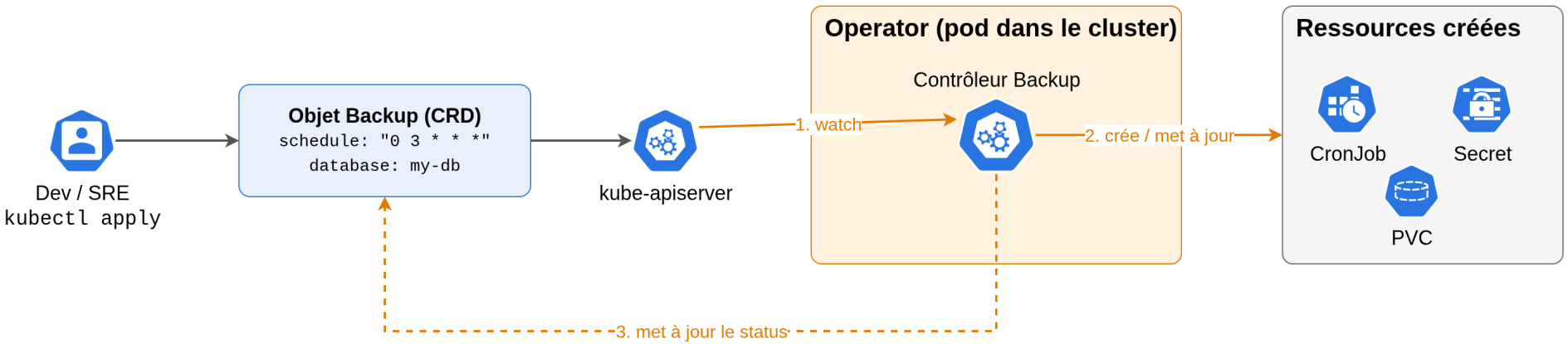
Les CRDs installées dans un cluster :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get crds
NAME                                  CREATED AT
applications.argoproj.io              2026-10-20T14:02:11Z
backups.example.com                   2026-10-22T09:12:44Z
certificates.cert-manager.io          2026-10-20T14:05:37Z
clusters.postgresql.cnpg.io           2026-10-21T08:41:02Z
$ kubectl apply -f nightly-backup.yaml
backup.example.com/nightly created
$ kubectl get backups
NAME      AGE
nightly   5s
```

Les **operators**:

- Operator = CRD + un contrôleur sur-mesure qui tourne dans le cluster
- Le contrôleur applique la boucle de réconciliation à nos propres objets
- Il encode le savoir-faire d'un humain (l'"*opérateur*"): installer, mettre à jour, sauvegarder, basculer, ...
- #AutomationFirst: le runbook devient du code



CRD (le quoi) + contrôleur (le comment) = operator

Exemple: une base PostgreSQL haute disponibilité avec CloudNativePG

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: my-db
spec:
  instances: 3
  storage:
    size: 10Gi
```

- L'operator crée les pods, la réplication, les Services, les sauvegardes et gère le failover

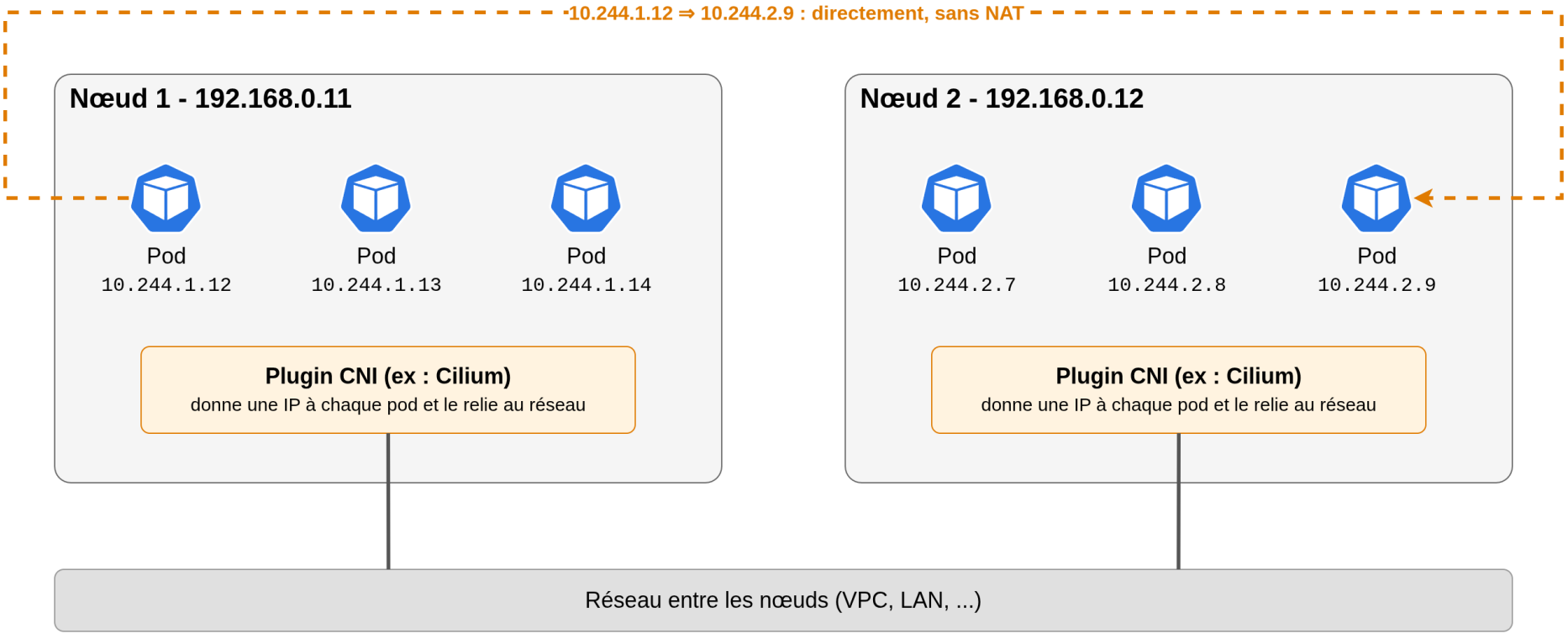
Quelques operators célèbres :

- **cert-manager**: certificats TLS automatiques (Let's Encrypt)
- **CloudNativePG**: PostgreSQL
- **Prometheus Operator**: le monitoring (`ServiceMonitor`, `PrometheusRule`)
- **Argo CD**: ses `Application` sont des CRDs
- **Crossplane**: gérer son infrastructure cloud depuis Kubernetes (Jour 1)

Attention: un operator, c'est aussi du code à maintenir, à mettre à jour et à surveiller

Le réseau dans Kubernetes:

- Chaque pod a sa propre IP
- Tous les pods peuvent se parler, même sur des nœuds différents, sans NAT
- Kubernetes définit ces règles... mais ne les implémente pas !
- C'est le rôle du plugin **CNI** (*Container Network Interface*), appelé à chaque création de pod


10.244.1.12 ⇒ 10.244.2.9 : directement, sans NAT
Nœud 1 - 192.168.0.11
Pod
10.244.1.12
Pod
10.244.1.13
Pod
10.244.1.14
Plugin CNI (ex : Cilium)
donne une IP à chaque pod et le relie au réseau
Nœud 2 - 192.168.0.12
Pod
10.244.2.7
Pod
10.244.2.8
Pod
10.244.2.9
Plugin CNI (ex : Cilium)
donne une IP à chaque pod et le relie au réseau
Réseau entre les nœuds (VPC, LAN, ...)

Quelques CNI:

- **Cilium**: basé sur eBPF, NetworkPolicies avancées, observabilité réseau (Hubble)
- **Calico**: très répandu, NetworkPolicies
- **Flannel**: simple... mais n'applique pas les NetworkPolicies !
- Les CNI des clouds (AWS VPC CNI, Azure CNI, ...): les pods ont des IPs du réseau du cloud

Les NetworkPolicies (section Sécurité) ne servent à rien si le CNI ne les applique pas

Le stockage dans Kubernetes:

- Le système de fichiers d'un conteneur est éphémère: il disparaît avec le pod
- **PersistentVolumeClaim** (PVC): *"j'ai besoin de 10 Go"* (côté dev)
- **PersistentVolume** (PV): le vrai disque (côté plateforme)
- **StorageClass**: le type de stockage (nombre de répliques, ...) et qui le crée
- Le driver **CSI** (*Container Storage Interface*) crée et attache le volume à la demande

Exemple de stockage CSI: **Longhorn**

- Un stockage distribué qui tourne **dans** le cluster (projet CNCF, s'installe avec Helm)
- Il utilise les disques locaux des nœuds (`/var/lib/longhorn`)
- Chaque volume est **répliqué** sur plusieurs nœuds (3 par défaut): un nœud tombe, les données restent
- Son driver CSI `driver.longhorn.io` crée les volumes à la demande
- Snapshots, sauvegardes vers S3 / NFS et une interface web pour tout voir

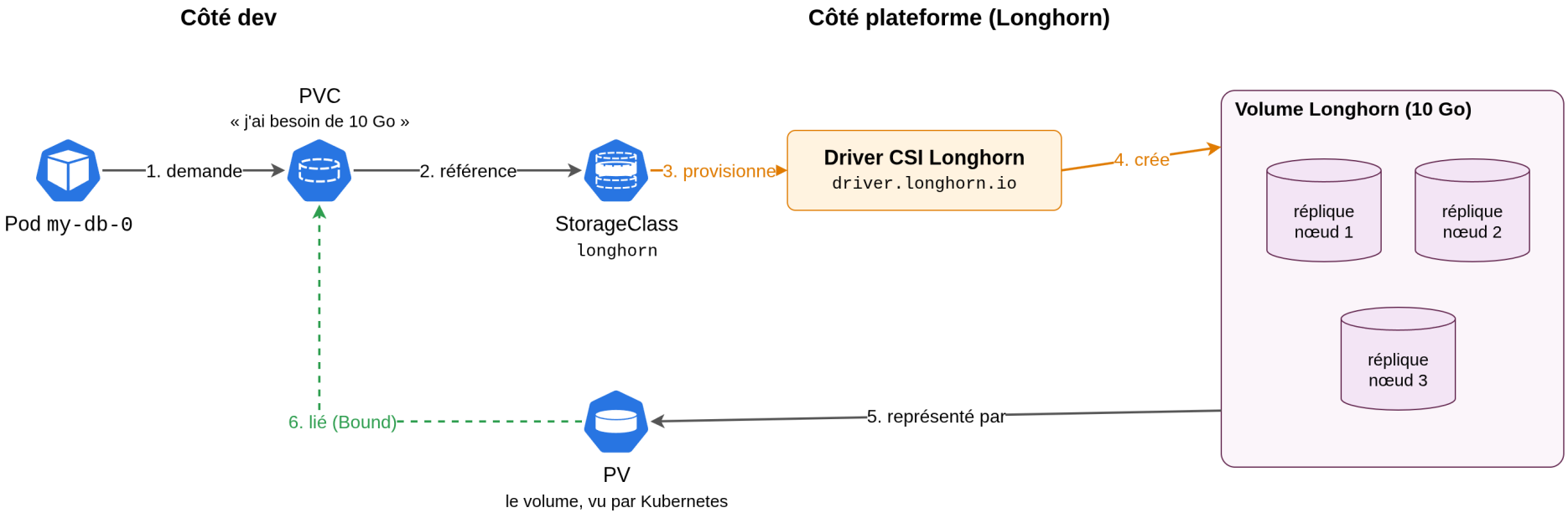
```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-my-db
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: longhorn
  resources:
    requests:
      storage: 10Gi
```

- `ReadWriteOnce`: monté par un seul nœud à la fois (le cas classique)
- `ReadWriteMany`: partagé entre plusieurs nœuds (Longhorn l'expose alors en NFS)

StorageClass, PVC et PV :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get storageclass
NAME                 PROVISIONER          RECLAIMPOLICY   VOLUMEBINDINGMODE   AGE
longhorn (default)   driver.longhorn.io   Delete          Immediate           12d
$ kubectl get pvc
NAME         STATUS   VOLUME                                     CAPACITY   STORAGECLASS
data-my-db   Bound    pvc-3f2a9c1e-8b4d-4e7a-9c5f-1d2e3f4a5b6c   10Gi       longhorn
$ kubectl get pv -o custom-
  columns=NAME:.metadata.name,SIZE:.spec.capacity.storage,CLAIM:.spec.claimRef.name
NAME                                       SIZE   CLAIM
pvc-3f2a9c1e-8b4d-4e7a-9c5f-1d2e3f4a5b6c   10Gi   data-my-db
```



7. le volume est monté dans le pod : les données survivent au pod... et à la perte d'un nœud

Le volume, vu par Longhorn (2 répliques: notre cluster n'a que 2 workers) :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl get volumes.longhorn.io -n longhorn-system -o custom-columns=NAME:.metadata.name,STATE:.status.state,ROBUSTNESS:.status.robustness,REPLICAS:.spec.numberOfReplicas
NAME                                       STATE      ROBUSTNESS   REPLICAS
pvc-3f2a9c1e-8b4d-4e7a-9c5f-1d2e3f4a5b6c   attached   healthy      2
$ kubectl get replicas.longhorn.io -n longhorn-system -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeID,STATE:.status.currentState
NAME                                                NODE           STATE
pvc-3f2a9c1e-8b4d-4e7a-9c5f-1d2e3f4a5b6c-r-1b2c3d4e   kind-worker    running
pvc-3f2a9c1e-8b4d-4e7a-9c5f-1d2e3f4a5b6c-r-5f6a7b8c   kind-worker2   running
```

Contrôler où vont les pods:

- `nodeSelector` / **node affinity**: ce pod doit aller sur un nœud avec un GPU, dans telle zone, ...
- **pod anti-affinity**: ne pas mettre deux replicas sur le même nœud
- **taints** / **tolerations**: réserver des nœuds (seuls les pods qui tolèrent le taint peuvent y aller)

Questions ?

Le problème :

- Deployment + Service + Ingress + HPA + ConfigMap + Secret + ...
  = beaucoup de YAML

Le problème :

- Deployment + Service + Ingress + HPA + ConfigMap + Secret + ... = beaucoup de YAML
- Les mêmes fichiers pour dev, staging et prod, à quelques valeurs près

Le problème :

- Deployment + Service + Ingress + HPA + ConfigMap + Secret + ... = beaucoup de YAML
- Les mêmes fichiers pour dev, staging et prod, à quelques valeurs près
- Copier-coller entre environnements $\Rightarrow$ *"configuration drift"*

Le problème :

- Deployment + Service + Ingress + HPA + ConfigMap + Secret + ... = beaucoup de YAML
- Les mêmes fichiers pour dev, staging et prod, à quelques valeurs près
- Copier-coller entre environnements ⇒ *"configuration drift"*
- Il nous faut un **gestionnaire de paquets** pour Kubernetes

Un **chart** Helm :

- Un paquet qui contient tous les objets Kubernetes d'une application
- `Chart.yaml`: le nom, la version du chart et de l'application
- `templates/`: les fichiers YAML avec des trous à remplir (templates Go)
- `values.yaml`: les valeurs par défaut pour remplir ces trous
- Versionné et publié dans un registre (comme une image)

Un chart simple :

```
my-chart/
|-- Chart.yaml
|-- values.yaml
`-- templates/
    |-- deployment.yaml
    `-- service.yaml
```

Chart.yaml :

```yaml
apiVersion: v2
name: my-chart
version: 0.1.0
appVersion: "1.0.0"
```

values.yaml :

```yaml
replicaCount: 2
image:
  repository: myapp
  tag: v1.0.0
service:
  port: 80
```

```
helm install my-app ./my-chart
```

Un chart plus complet, avec des valeurs par environnement :

```
my-chart/
|-- CHANGELOG
|-- Chart.yaml
|-- README.md
|-- templates
|   |-- _helpers.tpl
|   |-- configmap.yaml
|   |-- deployment.yaml
|   |-- externalsecrets.yaml
|   |-- hpa.yaml
|   |-- ingress.yaml
|   |-- service.yaml
|   |-- serviceaccount.yaml
|   `-- servicemonitor.yaml
|-- values.yaml
|-- values-test.yaml
`-- values-prod.yaml
```

values-prod.yaml :

```yaml
replicaCount: 3
image:
  tag: v1.2.0
ingress:
  enabled: true
  host: www.example.com
autoscaling:
  enabled: true
  maxReplicas: 10
```

```
helm upgrade my-app ./my-chart \
  -f values-prod.yaml
```

templates/deployment.yaml :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-chart.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

values-prod.yaml :

```yaml
replicaCount: 3
image:
  repository: myapp
  tag: v1.0.0
```

Les commandes du quotidien :

```
# Installer un chart depuis un registre OCI
helm install nginx oci://ghcr.io/nginx/charts/nginx-ingress \
  --version 2.3.0
# Mettre à jour avec ses propres valeurs
helm upgrade my-app ./my-chart -f values-prod.yaml
# Voir ce qui est installé dans le cluster
helm list -A
# Revenir à la révision précédente
helm rollback my-app 1
```

```
$ helm list -A
NAME                NAMESPACE          REVISION  STATUS    CHART                      APP VERSION
argocd              argocd             1         deployed  argocd-8.0.10
cilium              kube-system        2         deployed  cilium-1.16.7              1.16.7
coredns             kube-system        1         deployed  coredns-1.39.1             1.12.0
external-secrets    external-secrets   1         deployed  external-secrets-0.12.1
metrics-server      kube-system        1         deployed  metrics-server-3.12.2      0.7.2
osc-bsu-csi-driver  kube-system        1         deployed  osc-bsu-csi-driver-1.7.0   v1.5.0
```

Helm en pratique :

- Toutes les briques du cluster s'installent avec Helm (ingress controller, monitoring, ArgoCD, ...)
- Chaque `install`/`upgrade` crée une **révision**: on peut revenir en arrière
- Un chart propre = des valeurs par environnement, pas des copies de YAML
- Alternative sans templates: **Kustomize** et ses *overlays* par environnement

1. Un (ou plusieurs) conteneur(s) qui package mon application pour la faire tourner ailleurs que sur ma machine

1. Un (ou plusieurs) conteneur(s) qui package mon application pour la faire tourner ailleurs que sur ma machine
2. Une plateforme extensible qui orchestre les conteneurs de mon application

1. Un (ou plusieurs) conteneur(s) qui package mon application pour la faire tourner ailleurs que sur ma machine
2. Une plateforme extensible qui orchestre les conteneurs de mon application
3. Une solution qui package les objets supplémentaires à mon application

1. Un (ou plusieurs) conteneur(s) qui package mon application pour la faire tourner ailleurs que sur ma machine
2. Une plateforme extensible qui orchestre les conteneurs de mon application
3. Une solution qui package les objets supplémentaires à mon application

Il reste une question: **qui** lance `helm upgrade` ? Et depuis où ?

Questions ?

Qui déploie ?

- Le dev, depuis son poste, avec `kubectl` ou `helm` ? $\Rightarrow$ qui a fait quoi, et quand ?

Qui déploie ?

- Le dev, depuis son poste, avec `kubectl` ou `helm` ? $\Rightarrow$ qui a fait quoi, et quand ?
- La CI, avec les accès admin du cluster ? $\Rightarrow$ des clés très puissantes dans la CI

Qui déploie ?

- Le dev, depuis son poste, avec `kubectl` ou `helm` ? $\Rightarrow$ qui a fait quoi, et quand ?
- La CI, avec les accès admin du cluster ? $\Rightarrow$ des clés très puissantes dans la CI
- Et si quelqu'un modifie le cluster à la main ? $\Rightarrow$ le cluster ne correspond plus à ce qui est dans git

Argo CD:

- Un outil de déploiement continu qui tourne **dans** le cluster
- Il surveille un dépôt git (YAML, chart Helm, Kustomize)
- Il applique ce qu'il y trouve dans le cluster (*sync*)
- Il détecte les écarts entre git et le cluster (*drift*) et peut les corriger (*self-heal*)
- Une interface web pour voir l'état de chaque application

Comment ça marche ?

- Argo CD s'installe dans le cluster (avec Helm !) dans son namespace `argocd`
- **repo-server**: clone le dépôt git et génère les manifests (`helm template`, Kustomize, YAML)
- **application-controller**: compare ces manifests avec le cluster et applique les différences
- **API server + UI**: l'interface web, la CLI `argocd`, l'authentification (SSO) et les droits
- C'est lui qui tire (*pull*) depuis git: personne ne pousse dans le cluster depuis l'extérieur

Les concepts d'Argo CD :

- **Application** (une CRD !): *"ce dossier de ce dépôt git"* → *"ce namespace de ce cluster"*
- **AppProject**: regroupe des Applications et limite ce qu'elles ont le droit de faire (dépôts, clusters, namespaces autorisés)
- **Sync status**: `Synced` (le cluster = git) ou `OutOfSync` (il y a un écart)
- **Health status**: `Healthy`, `Progressing`, `Degraded`, ... (les pods sont-ils prêts ?)
- **Historique**: chaque sync est enregistré, on peut revenir à une révision précédente

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/my-org/my-app-deploy.git
    targetRevision: main
    path: charts/my-app
    helm:
      valueFiles: [values-prod.yaml]
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app
  syncPolicy:
    automated: {prune: true, selfHeal: true}
```

La **sync policy** :

- **Manuelle**: Argo CD signale les écarts, un humain clique sur *Sync*
- **Automatique**: chaque commit sur la branche suivie est déployé
- `prune`: supprime du cluster ce qui a été supprimé de git
- `selfHeal`: annule les modifications faites à la main dans le cluster
- *Sync waves* et *hooks*: ordonner le déploiement (ex: un Job de migration de base avant le Deployment)

L'état d'une Application :

elixir@sandbox: ~ (kind-sandbox)

```
$ argocd app get my-app
Name:               argocd/my-app
Project:            default
Server:             https://kubernetes.default.svc
Namespace:          my-app
Source:
- Repo:             https://github.com/my-org/my-app-deploy.git
  Target:           main
  Path:             charts/my-app
  Helm Values:      values-prod.yaml
Sync Policy:        Automated (Prune)
Sync Status:        Synced to main (3f9c2a1)
Health Status:      Healthy

GROUP  KIND        NAMESPACE  NAME           STATUS  HEALTH
       ConfigMap   my-app     my-config      Synced
       Service     my-app     my-service     Synced  Healthy
apps   Deployment  my-app     my-deployment  Synced  Healthy
```

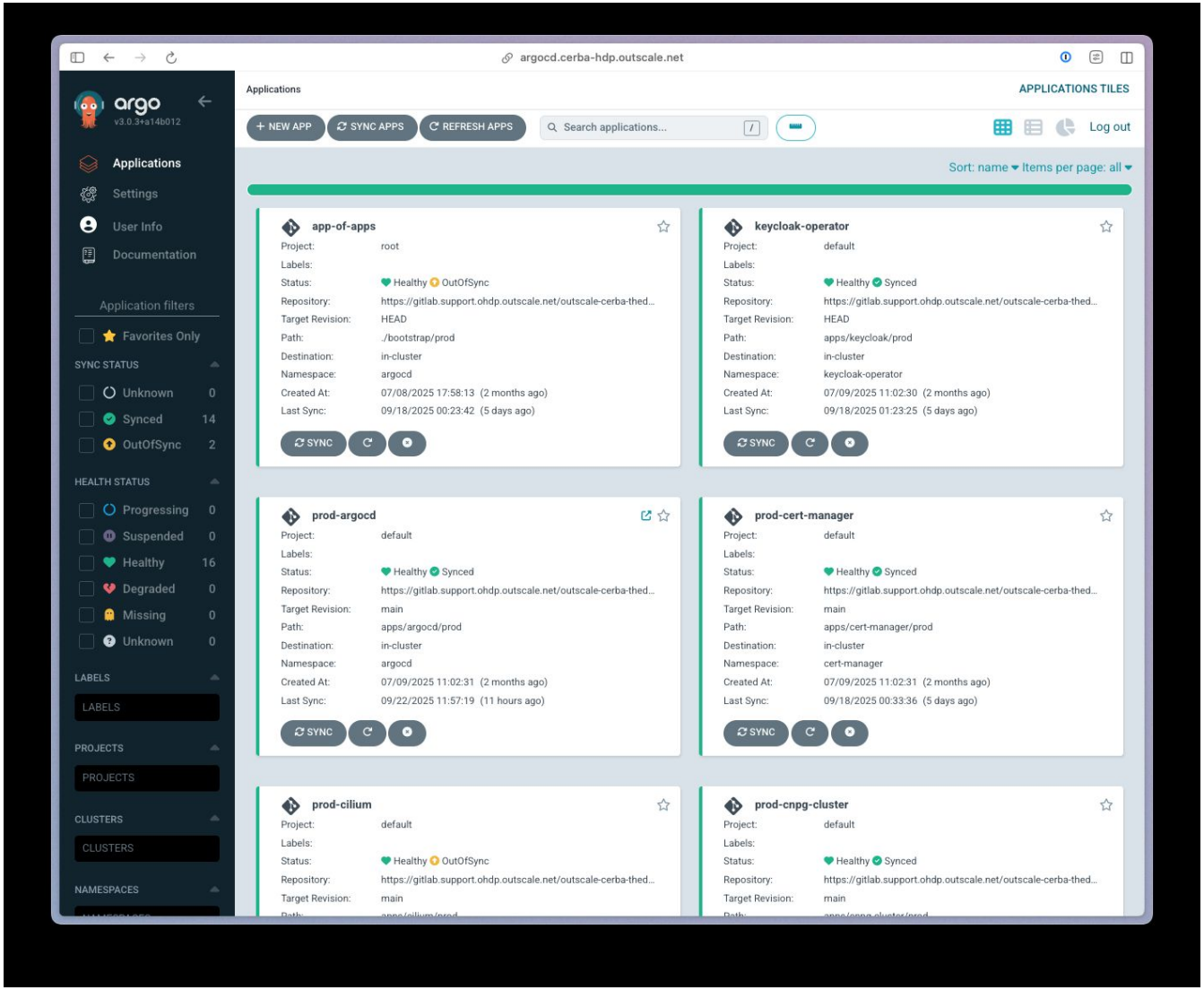
Une modification à la main ? Le `selfHeal` la corrige :

elixir@sandbox: ~ (kind-sandbox)

```
$ kubectl scale deployment/my-deployment -n my-app --replicas=5
deployment.apps/my-deployment scaled
$ argocd app get my-app | grep Status
Sync Status:        OutOfSync from main (3f9c2a1)
Health Status:      Healthy
$ sleep 10   # self-heal
$ argocd app get my-app | grep Status
Sync Status:        Synced to main (3f9c2a1)
Health Status:      Healthy
$ kubectl get deployment my-deployment -n my-app
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
my-deployment   3/3     3            3           2d
```

Passer à l'échelle :

- **App of Apps**: une Application qui déploie d'autres Applications (tout le cluster décrit dans git)
- **ApplicationSet**: générer une Application par environnement, par cluster ou par dossier du dépôt
- Un seul Argo CD peut piloter plusieurs clusters (dev, staging, prod)
- **Notifications**: prévenir sur Slack, mail, ... quand un sync échoue ou qu'une app devient `Degraded`


argocd.cerba-hdp.outscale.net
argo
v3.0.3+a14b012
Applications
Settings
User Info
Documentation
Application filters
Favorites Only
SYNC STATUS
Unknown 0
Synced 14
OutOfSync 2
HEALTH STATUS
Progressing 0
Suspended 0
Healthy 16
Degraded 0
Missing 0
Unknown 0
LABELS
PROJECTS
CLUSTERS
NAMESPACES
Applications
APPLICATIONS TILES
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
Log out
Sort: name Items per page: all
app-of-apps
Project: root
Labels:
Status: Healthy OutOfSync
Repository: https://gitlab.support.ohdp.outscale.net/outscale-cerba-thed...
Target Revision: HEAD
Path: ./bootstrap/prod
Destination: in-cluster
Namespace: argocd
Created At: 07/08/2025 17:58:13 (2 months ago)
Last Sync: 09/18/2025 00:23:42 (5 days ago)
SYNC
keycloak-operator
Project: default
Labels:
Status: Healthy Synced
Repository: https://gitlab.support.ohdp.outscale.net/outscale-cerba-thed...
Target Revision: HEAD
Path: apps/keycloak/prod
Destination: in-cluster
Namespace: keycloak-operator
Created At: 07/09/2025 11:02:30 (2 months ago)
Last Sync: 09/18/2025 01:23:25 (5 days ago)
SYNC
prod-argocd
Project: default
Labels:
Status: Healthy Synced
Repository: https://gitlab.support.ohdp.outscale.net/outscale-cerba-thed...
Target Revision: main
Path: apps/argocd/prod
Destination: in-cluster
Namespace: argocd
Created At: 07/09/2025 11:02:31 (2 months ago)
Last Sync: 09/22/2025 11:57:19 (11 hours ago)
SYNC
prod-cert-manager
Project: default
Labels:
Status: Healthy Synced
Repository: https://gitlab.support.ohdp.outscale.net/outscale-cerba-thed...
Target Revision: main
Path: apps/cert-manager/prod
Destination: in-cluster
Namespace: cert-manager
Created At: 07/09/2025 11:02:31 (2 months ago)
Last Sync: 09/18/2025 00:33:36 (5 days ago)
SYNC
prod-cilium
Project: default
Labels:
Status: Healthy OutOfSync
Repository: https://gitlab.support.ohdp.outscale.net/outscale-cerba-thed...
Target Revision: main
prod-cnpg-cluster
Project: default
Labels:
Status: Healthy Synced
Repository: https://gitlab.support.ohdp.outscale.net/outscale-cerba-thed...
Target Revision: main

Le **GitOps**: 4 principes

- **Déclaratif**: l'état désiré du système est décrit, pas les étapes pour y arriver

Le **GitOps**: 4 principes

- **Déclaratif**: l'état désiré du système est décrit, pas les étapes pour y arriver
- **Versionné et immuable**: cet état est stocké dans git, avec tout l'historique

Le **GitOps**: 4 principes

- **Déclaratif**: l'état désiré du système est décrit, pas les étapes pour y arriver
- **Versionné et immuable**: cet état est stocké dans git, avec tout l'historique
- **Tiré automatiquement**: des agents récupèrent cet état (*pull*), personne ne le pousse

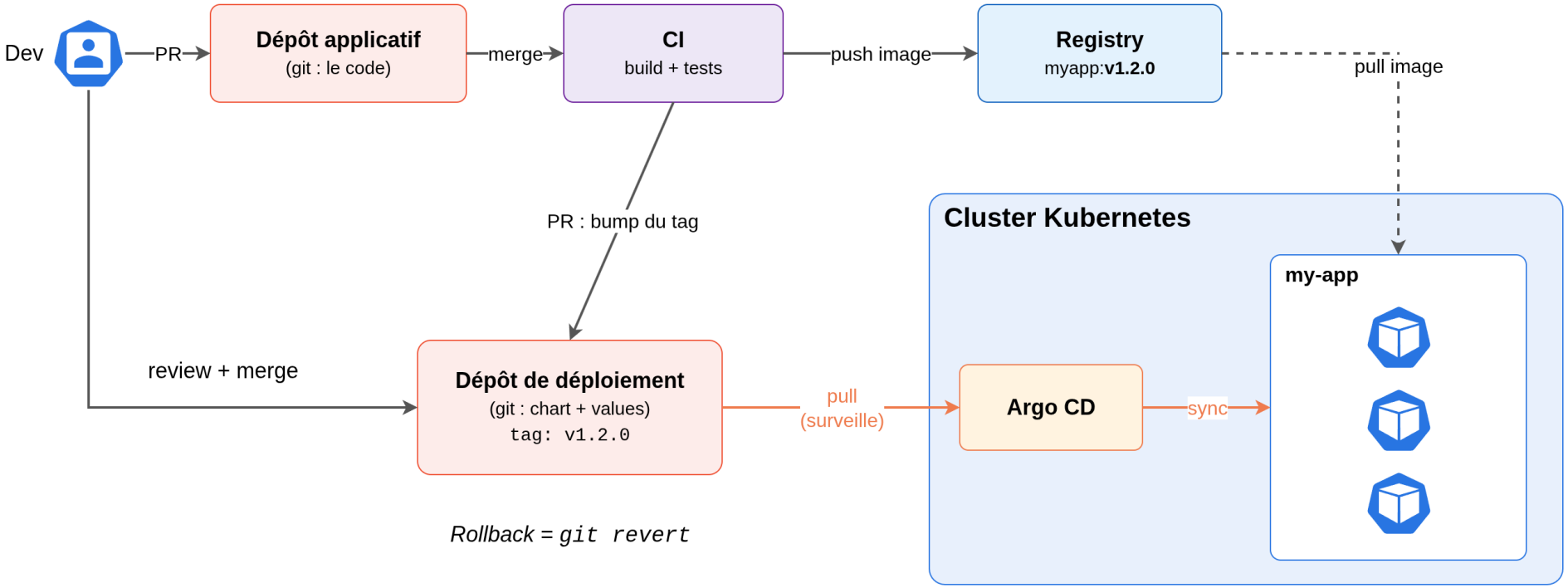
Le **GitOps**: 4 principes

- **Déclaratif**: l'état désiré du système est décrit, pas les étapes pour y arriver
- **Versionné et immuable**: cet état est stocké dans git, avec tout l'historique
- **Tiré automatiquement**: des agents récupèrent cet état (*pull*), personne ne le pousse
- **Réconcilié en continu**: les agents corrigent tout écart

Le **GitOps**: 4 principes

- **Déclaratif**: l'état désiré du système est décrit, pas les étapes pour y arriver
- **Versionné et immuable**: cet état est stocké dans git, avec tout l'historique
- **Tiré automatiquement**: des agents récupèrent cet état (*pull*), personne ne le pousse
- **Réconcilié en continu**: les agents corrigent tout écart

La boucle de réconciliation de Kubernetes... appliquée au dépôt git !


Dev
PR
Dépôt applicatif
(git : le code)
merge
CI
build + tests
push image
Registry
myapp:v1.2.0
pull image
PR : bump du tag
Cluster Kubernetes
my-app
review + merge
Dépôt de déploiement
(git : chart + values)
tag: v1.2.0
pull
(surveille)
Argo CD
sync
Rollback = git revert

Pourquoi c'est génial :

- Un déploiement = une Pull Request (review, historique, traçabilité)
- Un rollback = un `git revert`
- Plus besoin de donner les clés du cluster à la CI ni aux devs
- Drift = 0: le cluster est toujours le reflet de git
- Même philosophie qu'Atlantis pour Terraform (Jour 1)
- Alternative: **Flux**

Attention aux pièges :

- Ne jamais réutiliser un tag d'image (*"retag sauvage"*): `v1.0.0` doit toujours désigner la même image
- Ne pas utiliser `latest`: impossible de savoir ce qui tourne
- Une modification à la main sera écrasée (ou signalée) par Argo CD: résoudre le drift dans git

Questions ?

Les enjeux :

- Tenir les SLOs malgré les pics
- Élasticité sans sur-provisionner
- MTTR ↓ : le système se répare seul
- Coût maîtrisé : right-sizing, autoscaling
- Sécurité : RBAC, NetworkPolicies

Les objets :

- Namespace : ranger le cluster
- Pod : l'unité de base
- Deployment, StatefulSet, DaemonSet, (Cron)Job
- Service + Ingress : exposer
- ConfigMap / Secret : la configuration
- HPA : scaler selon la charge

Le cluster :

- Des nœuds enrôlés = un réservoir de capacité
- Control plane : api-server, etcd, scheduler, controllers
- Workers : kubelet, kube-proxy, runtime
- Déclaratif + boucle de réconciliation
- Extensible : CRDs + operators
- Délègue : CNI (réseau), CSI (stockage, Longhorn)

Les bonnes pratiques :

- Probes : liveness / readiness
- Requests / limits
- NetworkPolicies + RBAC minimal
- Pas de secret en clair
- Applications stateless + HPA
- Exposer /metrics

Le déploiement :

- Kustomize : base + overlays
- Helm : chart + values par environnement
- Argo CD : sync git ⇒ cluster, selfHeal
- GitOps : un déploiement = une PR
- Rollback = git revert, drift = 0
- Jamais de retag, jamais de latest

Promesse: tenir les SLOs quand la demande explose, automatiquement et au meilleur coût.

- **Auto-healing** (probes) avant autoscaling
- **HPA** pour suivre la charge, sur des applications **stateless**
- **Requests** = SLA de scheduling, **limits** = garde-fous
- **GitOps** pour des changements répétables et réversibles
- Les **SLO** pilotent la stratégie de scaling (latence, erreurs, saturation)

Pièges à éviter :

- HPA sans requests $\Rightarrow$ métriques $\neq$ réalité
- Tout résoudre par le CPU (ignorer I/O, connexions, DB)
- Limits trop serrées $\Rightarrow$ OOMKill
- Pas de readiness probe $\Rightarrow$ du trafic envoyé à des pods pas prêts
- Retag sauvage / drift GitOps non résolu

Checklist :

- Probes, requests/limits, HPA
- Service + Ingress opérationnels
- Config dans des ConfigMaps/Secrets, NetworkPolicies, RBAC minimal
- Chart Helm propre / overlays Kustomize par environnement
- Application synchronisée (Argo CD), drift = 0

Questions ?

Quelques sources :

[Ver+15] Abhishek Verma et al. 'Large-scale cluster management at Google with Borg'. In: *Proceedings of the European Conference on Computer Systems (EuroSys)*. 2015. URL: https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/.

[Bur+16] Brendan Burns et al. 'Borg, Omega, and Kubernetes'. In: *ACM Queue* 14.1 (2016). URL: https://queue.acm.org/detail.cfm?id=2898444.

[Arg] Argo Project. *Argo CD - Declarative GitOps CD for Kubernetes*. URL: https://argo-cd.readthedocs.io/en/stable/. (accessed: October 22, 2026).

[cer] cert-manager. *cert-manager Documentation*. URL: https://cert-manager.io/docs/. (accessed: October 22, 2026).

[Cil] Cilium. *Cilium Documentation*. URL: https://docs.cilium.io/en/stable/. (accessed: October 22, 2026).

[Clo] CloudNativePG. *CloudNativePG Documentation*. URL: https://cloudnative-pg.io/documentation/current/. (accessed: October 22, 2026).

[Ext] External Secrets. *External Secrets Operator*. URL: https://external-secrets.io/. (accessed: October 22, 2026).

[Flu] Flux. *Flux - the GitOps family of projects*. URL: https://fluxcd.io/. (accessed: October 22, 2026).

[Hel] Helm. *Helm Documentation*. URL: https://helm.sh/docs/. (accessed: October 22, 2026).

[Kub] Kubernetes SIG CLI. *Kustomize*. URL: https://kustomize.io/. (accessed: October 22, 2026).

[Lon] Longhorn. *Longhorn Documentation*. URL: https://longhorn.io/docs/. (accessed: October 22, 2026).

[Ope] OpenGitOps. *OpenGitOps Principles*. URL: https://opengitops.dev/. (accessed: October 22, 2026).

[Pro] Prometheus. *Prometheus - Monitoring system and time series database*. URL: https://prometheus.io/docs/introduction/overview/. (accessed: October 22, 2026).

[K8sa] The Kubernetes Authors. *Assigning Pods to Nodes*. URL: https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/. (accessed: October 22, 2026).

[K8sb] The Kubernetes Authors. *ConfigMaps*. URL: https://kubernetes.io/docs/concepts/configuration/configmap/. (accessed: October 22, 2026).

[K8sc] The Kubernetes Authors. *Configure a Security Context for a Pod or Container*. URL: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/. (accessed: October 22, 2026).

[K8sd] The Kubernetes Authors. *Configure Liveness, Readiness and Startup Probes*. URL: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/. (accessed: October 22, 2026).

[K8se] The Kubernetes Authors. *Custom Resources*. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/. (accessed: October 22, 2026).

[K8sf] The Kubernetes Authors. *Deployments*. URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/. (accessed: October 22, 2026).

[K8sg] The Kubernetes Authors. *Gateway API*. URL: https://gateway-api.sigs.k8s.io/. (accessed: October 22, 2026).

[K8sh] The Kubernetes Authors. *Horizontal Pod Autoscaling*. URL: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/. (accessed: October 22, 2026).

[K8si] The Kubernetes Authors. *Ingress*. URL: https://kubernetes.io/docs/concepts/services-networking/ingress/. (accessed: October 22, 2026).

[K8sj] The Kubernetes Authors. *Kubernetes Components*. URL: https://kubernetes.io/docs/concepts/overview/components/. (accessed: October 22, 2026).

[K8sk] The Kubernetes Authors. *Kubernetes Documentation*. URL: https://kubernetes.io/docs/home/. (accessed: October 22, 2026).

[K8sl] The Kubernetes Authors. *Namespaces*. URL: https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/. (accessed: October 22, 2026).

[K8sm] The Kubernetes Authors. *Network Plugins*. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/. (accessed: October 22, 2026).

[K8sn] The Kubernetes Authors. *Network Policies*. URL: https://kubernetes.io/docs/concepts/services-networking/network-policies/. (accessed: October 22, 2026).

[K8so] The Kubernetes Authors. *Operator pattern*. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/operator/. (accessed: October 22, 2026).

[K8sp] The Kubernetes Authors. *Overview - Kubernetes*. URL: https://kubernetes.io/docs/concepts/overview/. (accessed: October 22, 2026).

[K8sq] The Kubernetes Authors. *Persistent Volumes*. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/. (accessed: October 22, 2026).

[K8sr] The Kubernetes Authors. *Pods*. URL: https://kubernetes.io/docs/concepts/workloads/pods/. (accessed: October 22, 2026).

[K8ss] The Kubernetes Authors. *Resource Management for Pods and Containers*. URL: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/. (accessed: October 22, 2026).

[K8st] The Kubernetes Authors. *Secrets*. URL: https://kubernetes.io/docs/concepts/configuration/secret/. (accessed: October 22, 2026).

[K8su] The Kubernetes Authors. *Service*. URL: https://kubernetes.io/docs/concepts/services-networking/service/. (accessed: October 22, 2026).

[K8sv] The Kubernetes Authors. *Services, Load Balancing, and Networking*. URL: https://kubernetes.io/docs/concepts/services-networking/. (accessed: October 22, 2026).

[K8sw] The Kubernetes Authors. *StatefulSets*. URL: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/. (accessed: October 22, 2026).

[K8sx] The Kubernetes Authors. *Using RBAC Authorization*. URL: https://kubernetes.io/docs/reference/access-authn-authz/rbac/. (accessed: October 22, 2026).

[K8sy] The Kubernetes Authors. *Workload Management*. URL: https://kubernetes.io/docs/concepts/workloads/controllers/. (accessed: October 22, 2026).